

Data Science

Prof. Dr. André Drews

Fachbereich Maschinenbau und Wirtschaft

01. Juni 2026

- nur für den Gebrauch innerhalb des Kurses -

Vorlesungskapitel 7

- Regression, Clustering -

Lernziele

- Anwendungskompetenz Regression - Grundlagen, Decision Trees, SVM, KNN, ANN
- Kompetenz im Bereich Clustering

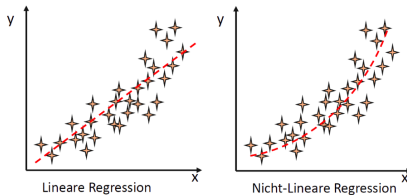
1. Regression
2. Clustering

- Decision trees und Entropie
- Bagging, Boosting, Stacking
- Random Forest
- AdaBoost
- Gradient Boosting
- XG-Boost
- LightGBM

1. Wiederholung des letzten Vorlesungskapitels
2. Regressionsverfahren
3. Clustering

Definition Regression

- Regression dient dazu den Zusammenhang $Y(X_i)$ zwischen der abhängigen Variablen Y und den unabhängigen Variablen X_i zu bestimmen.
- Die abhängige Variable Y ist bei der Regression immer kontinuierlich. Die unabhängigen Variablen können kontinuierlich oder diskret sein.

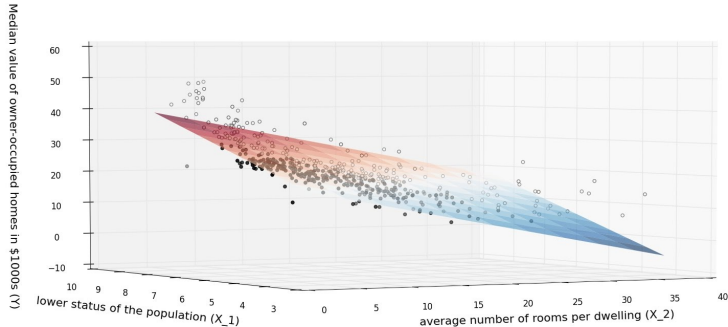


Quelle: <https://data-science-blog.com/blog/2017/12/20/maschinelles-lernen-klassifikation-vs-regression/> , Mai 2020

- **Prognose:** Wenn das Ziel die Vorhersage ist, dann kann der durch das Regressionsverfahren ermittelte funktionale Zusammenhang verwendet werden, um ein Vorhersagemodell zu erstellen. Wenn nun zusätzliche Werte x ohne zugehörigen Wert y vorliegen, können diese Werte vorhergesagt werden.
- **Zusammenhangsanalyse:** Wenn eine Variable y und eine Anzahl von Variablen vorliegen, die mit y in Verbindung gebracht werden können, dann können Regressionsverfahren angewandt werden, um die Stärke des Zusammenhangs zwischen y und x zu quantifizieren.

Boston Housing Price Prediction - Multivariate Regression

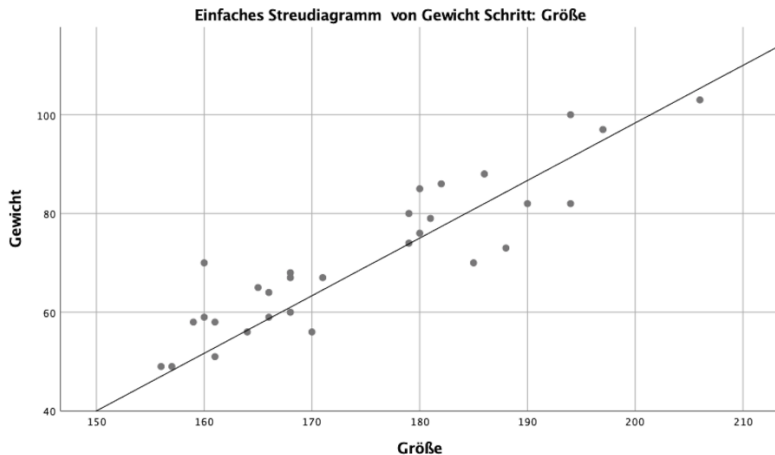
Multivariable Linear Regression: Q^2 : 0.64; R^2 : 0.69; p-value= 5.52e-235



Quelle: <https://sites.google.com/site/2015pcsu/data-science/boston-housing-pred>, Februar 2020

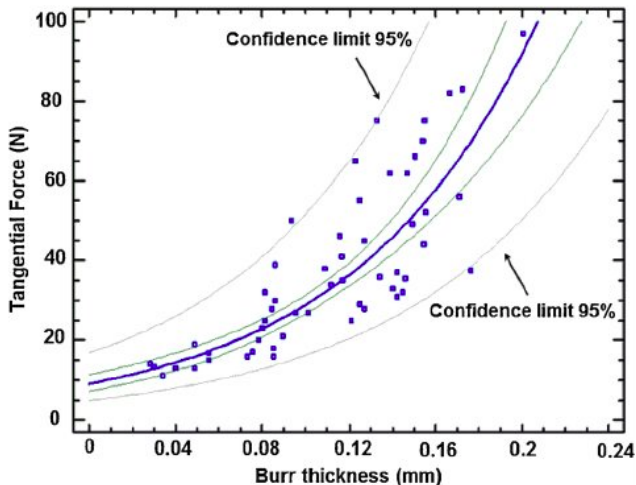
- Zielvariable MEDV: Median value of owner-occupied homes in \$1000s
- unabhängige Variable LSTAT: Percentage of lower status of the population
- unabhängige Variable RM: Average number of rooms per dwelling

Beispiel Lineare Regression



Quelle: <https://www.scribbr.de/statistik/regressionsanalyse/>, Februar 2020

Beispiel Exponential Regression

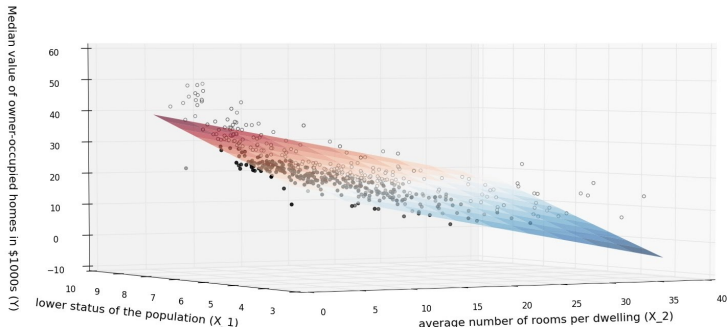


Quelle:

https://www.researchgate.net/publication/263050659i_Analytical_modelling_of_slot_milling_exit_burr_size/figures?lo=1,

Boston Housing Price Prediction - Multiple Regression

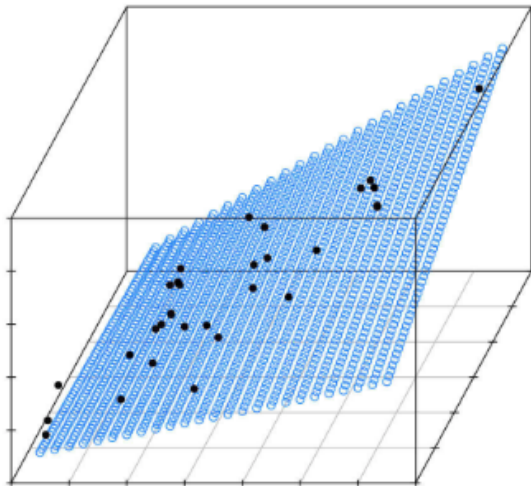
Multivariable Linear Regression: Q^2 : 0.64; R^2 : 0.69; p-value= 5.52e-235



Quelle: <https://sites.google.com/site/2015pcsu/data-science/boston-housing-pred>, Februar 2020

- Zielvariable MEDV: Median value of owner-occupied homes in \$1000s
- unabhängige Variable LSTAT: Percentage of lower status of the population
- unabhängige Variable RM: Average number of rooms per dwelling

Multiple Regression

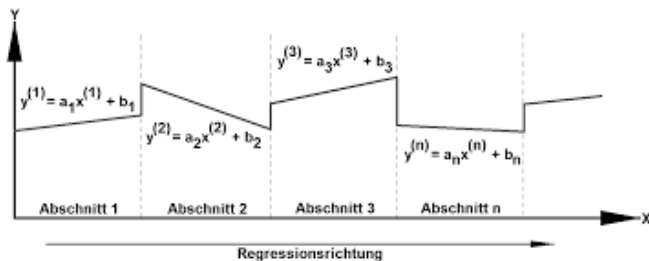


Quelle: <https://www.dataquest.io/blog/statistical-learning-for-predictive-modeling-r/>, März 2020

$$y = \beta_1 x_1 + \beta_2 x_2 + \alpha_1 + \alpha_2$$

- Skalenniveau der Zielvariable: metrisch/stetig
- Skalenniveau der unabhängigen Variablen: beliebig
- Homoskedastizität: Liegt keine Homoskedastizität vor, ist die Genauigkeit von $f(x)$ abhängig von x , was zu Ungenauigkeiten der Prognose führt.
- keine Ausreißer: Ausreißer können einen signifikanten Trend $f(x)$ verfälschen.

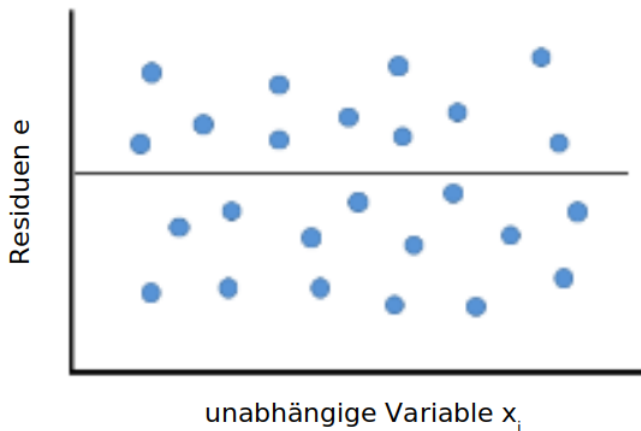
Voraussetzung für Regression: Stückweise Steitigkeit der Zielfunktion



Quelle: http://www.nadirpoint.de/Stueckweise_lineare_Regression.pdf

Voraussetzung: Homoskedastizität

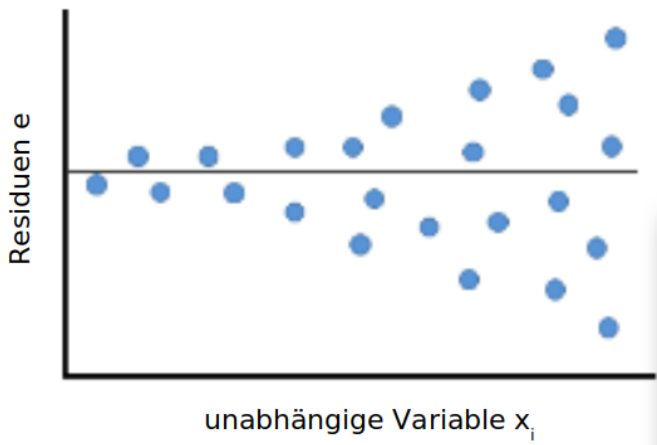
- Homoskedastizität: Die Varianz der Störterme ist konstant.



Quelle: <https://wikis.fu-berlin.de/display/fustat/Residuenplots>, Februar 2020

Gegenteil der Voraussetzung: Heterokedastizität

- Heteroskedastizität: Die Varianz der Störterme variiert.



Quelle: <https://wikis.fu-berlin.de/display/fustat/Residuenplots>, Februar 2020

- **Homoskedastizität:** Die Varianz der Residuen ist gleichmäßig verteilt. Falls dies nicht der Fall ist, gibt es für unterschiedliche Wertebereiche unterschiedliche Vorhersagegenauigkeiten der Regression.
- **Grobe Normalverteilung der Residuen:** Normalverteilte Residuen erlauben eine validere Interpretation der Ergebnisse als anders verteilte Residuen. Wären die Residuen z.B. asymmetrisch verteilt, würde dies zu Ungenauigkeiten der Bestimmung von $f(X)$ führen.
- **keine Multikolarität:** wenn zwei oder mehr unabhängige Variablen/Prädiktoren stark miteinander korrelieren, wird die Berechnung der Regressionskoeffizienten erschwert. Die Interpretation des Modells ist nicht mehr eindeutig.

Mathematische Beschreibung der einfachen Regression (1 Dimension, d.h. 1 unabhängige Variable)

Regression

- Die eindimensionale *Regression* beschreibt den Zusammenhang zwischen Zielvariablen Y und unabhängigen Variablen x über

$$\underbrace{Y}_{\text{Zielvariable}} = \underbrace{f(x)}_{\text{Approximation durch Regression}} + \underbrace{e}_{\text{Residuum}}$$

- Ziel ist eine optimale Approximation durch $f(X)$ an Y und damit einen möglichst geringen Fehler ϵ zu erreichen. Dies kann dadurch erzielt werden, daß das Residuum $\epsilon = Y - f(X)$ minimiert wird.

- Die Wahl der Funktion $f(x)$ entscheidet über die Art der Regression.
- Am häufigsten verbreitet ist die lineare Regression

$$f(x) = ax + b$$

- Als weitere Regressionsformen sind beliebige Funktionen $f(x)$ möglich wie

$$f(x) = Ax^2 + Bx + c,$$

$$f(x) = Ae^{bx},$$

...

Einfache Lineare Regression

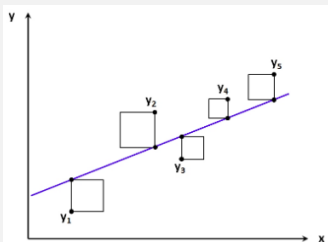
Die Regressionsfunktion hat die Form

$$f(X) = \alpha + \beta X$$

Für die Datenpaare (x_i, y_i) gilt dann die Beziehung:

$$y_i = f(X) + \epsilon_i = \alpha + \beta x_i + \epsilon_i, \quad i \in \{1, \dots, m\}$$

mit dem Achsenabschnitt α und der Steigung β .



Minimierung des Residuums:

$$R = \frac{1}{n} \sum_{i=1}^n |y_i - f(x_i)|^2$$

Quelle: <https://www.methodenberatung.uzh.ch>, Oktober 2019

Minimierung des Residuums: Kleinste Quadrate zur Herleitung der Parameter α und β

Analytische Minimierung des Residuums:

$$\frac{\partial R(\alpha, \beta)}{\partial \alpha} = \frac{2}{n} \sum_{i=1}^n \frac{y_i - (\alpha + \beta x)}{|y_i - (\alpha + \beta x)|} = 0, \quad \frac{\partial R(\alpha, \beta)}{\partial \beta} = \frac{2}{n} \sum_{i=1}^n \frac{y_i - (\alpha + \beta x)}{|y_i - (\alpha + \beta x)|} \cdot -x = 0 \quad (1)$$

ergibt

$$\bar{y} - \alpha - \beta \bar{x} = 0 \implies \alpha = \bar{y} - \beta \bar{x} \quad (2)$$

mit

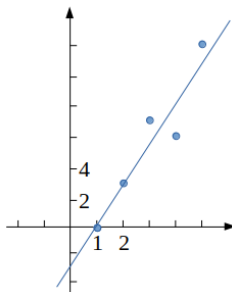
$$\beta = \frac{\sum_{i=1}^n y_i x_i - n \bar{y} \bar{x}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{S_{XY}}{S_X^2} \quad (3)$$

$$y = \alpha + \beta x$$

Beispiel Lineare Regression

x	1	2	3	4	5
y	0	3	7	6	12

- $\tilde{x} = 3$
- $\tilde{y} = 5.6$
- $\beta = \frac{\sum_{i=1}^5 (x_i - \tilde{x})(y_i - \tilde{y})}{\sum_{i=1}^5 (x_i - \tilde{x})^2} =$
$$\frac{(-2) \cdot (-5.6) + (-1) \cdot (3 - 2.6) + (1) \cdot (6 - 5.6) + (2) \cdot (12 - 5.6)}{(-2)^2 + (-1)^2 + 0 + (1)^2 + (2)^2} = \frac{27}{10} = 2.7$$
- $\alpha = 5.6 - 2.7 \cdot 3 = -2.5$



Die Sum of Squares Total (*SQT*: Summe der Quadrate der Abweichung der Beobachtungen vom Mittelwert der Beobachtungen) läßt sich in

Sum of Squares Explained (*SQE*: Summe der Abweichungen der Regressionspunkte vom Mittelwert der Beobachtungen) und

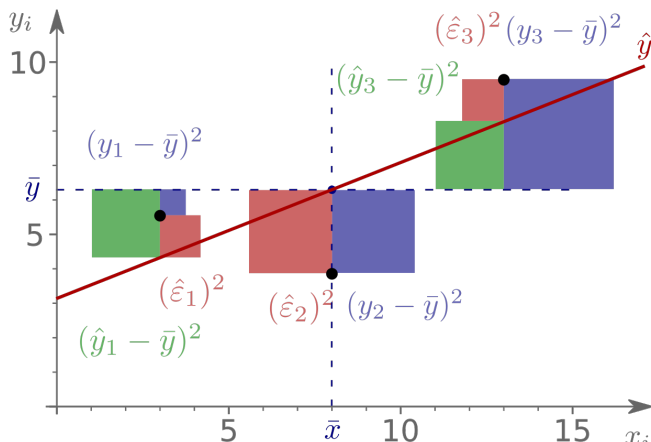
die Sum of Squares Residuals (*SQR*: Summe der Abweichungen der Regressionpunkte von den Beobachtungen) zerlegen.

Streuungszerlegung

$$SQT = SQE + SQR$$

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 + \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 + \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Lineare Regression

Das *Bestimmtheitsmaß* ist ein Maß dafür wie gut das Regressionsmodell die Werte approximiert.

Es ist definiert als

$$R^2 = \frac{SQE}{SQT} = \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{SQR}{SQT} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4)$$

mit

$$R^2 \in [0, 1]$$

Bestimmtheitsmaß und Bravais-Pearson Zusammenhangsmass

Zusammenhang

Es besteht ein Zusammenhang zwischen R^2 und dem Korrelationskoeffizienten nach Bravais-Pearson:

$$R^2 = r_{XY}^2$$

Herleitung:

$$\begin{aligned} R^2 &= \frac{\sum_{i=1}^n (\hat{y}_i - \tilde{y})^2}{\sum_{i=1}^n (y_i - \tilde{y})^2} = \frac{\beta^2 \sum_{i=1}^n (x_i - \bar{x})^2}{\sum_{i=1}^n (y_i - \tilde{y})^2} = \\ &= \frac{s_{XY}^2 \cdot s_X^2}{s_X^2 \cdot s_Y^2} = \left(\frac{s_{xy}}{s_X s_Y} \right)^2 = r_{xy}^2 \end{aligned} \quad (5)$$

→ wichtig für Zusammenhangsanalyse

Korrelationskoeffizient nach Bravais-Pearson

Der *Bravais-Pearson-Korrelationskoeffizient* mit den Wertepaaren (x_i, y_i) , $i = 1, \dots, n$

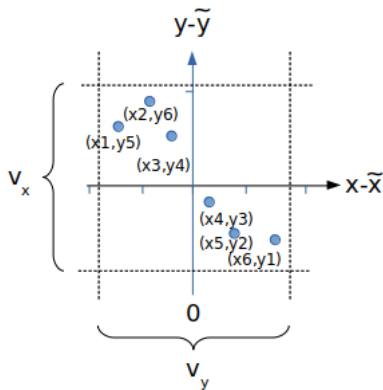
$$r = \frac{\sum_{i=1}^n (x_i - \tilde{x})(y_i - \tilde{y})}{\sqrt{\sum_{i=1}^n (x_i - \tilde{x})^2 \sum_{i=1}^n (y_i - \tilde{y})^2}}$$

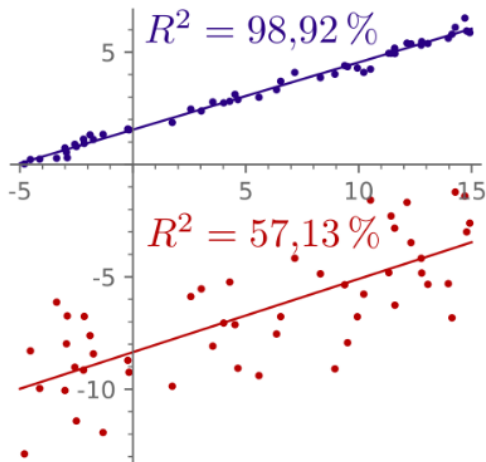
liegt in dem Wertebereich: $r \in [-1, 1]$.

Zwei Merkmale gelten als korreliert, falls $r \neq 0$ mit $r < 0$ als "negativ korreliert" (Steigung ist antiproportional) und $r > 0$ als "positiv korreliert" (Steigung ist proportional).

Veranschaulichung Korrelationskoeffizient nach Bravais-Pearson

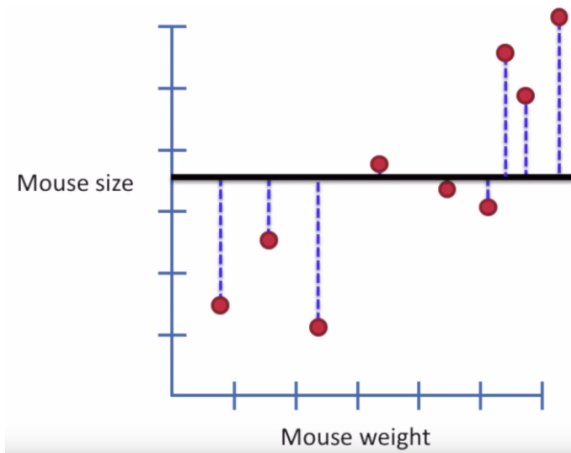
$$r = \frac{\sum_{i=1}^n (x_i - \tilde{x})(y_i - \tilde{y})}{\sqrt{\sum_{i=1}^n (x_i - \tilde{x})^2 \sum_{i=1}^n (y_i - \tilde{y})^2}} = \frac{\text{cov}(x,y)}{\sqrt{v_x \cdot v_y}}$$





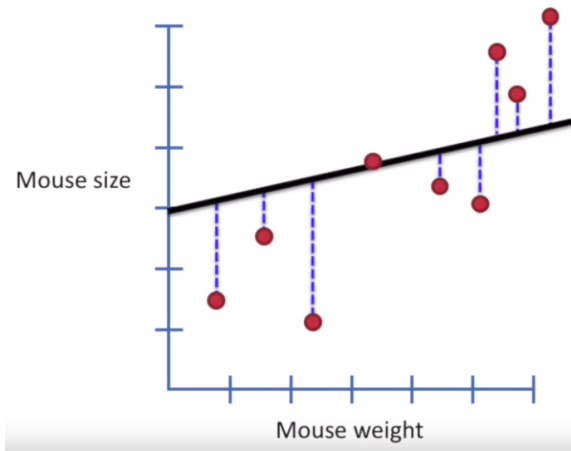
Quelle: wikipedia.org, Februar 2020

Lineare Regression - Datenset, fit der Regressionsgerade anschaulich



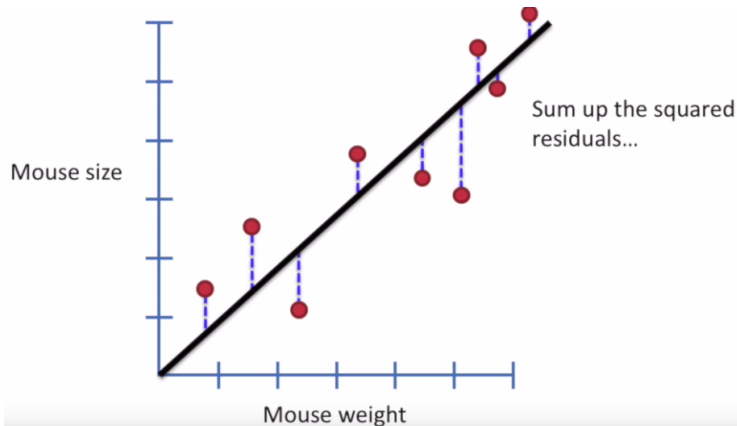
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Veränderung der Steigung anschaulich



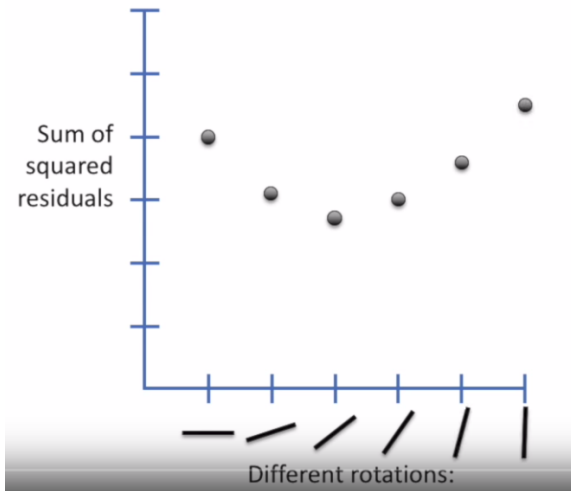
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Veränderung der Steigung anschaulich



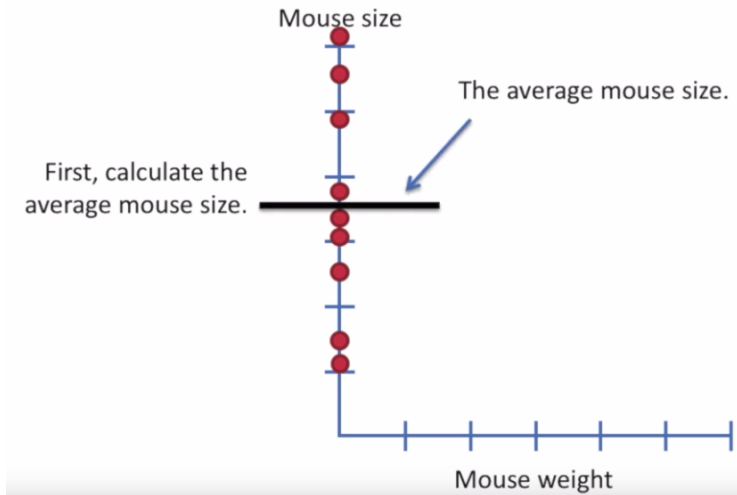
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Veränderung der Steigung anschaulich

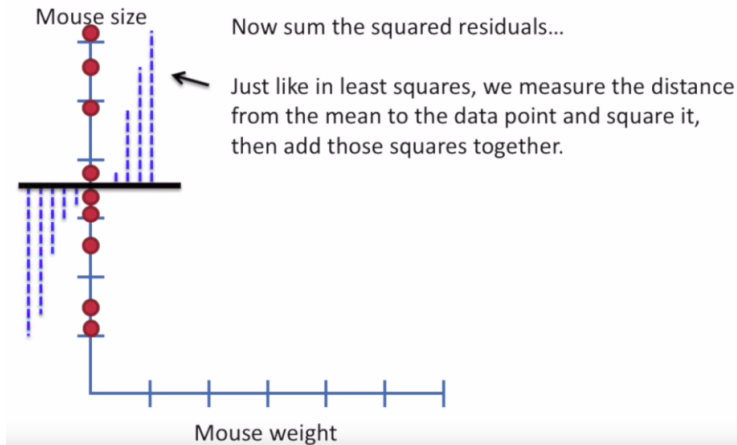


Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Variation um den Mittelwert anschaulich

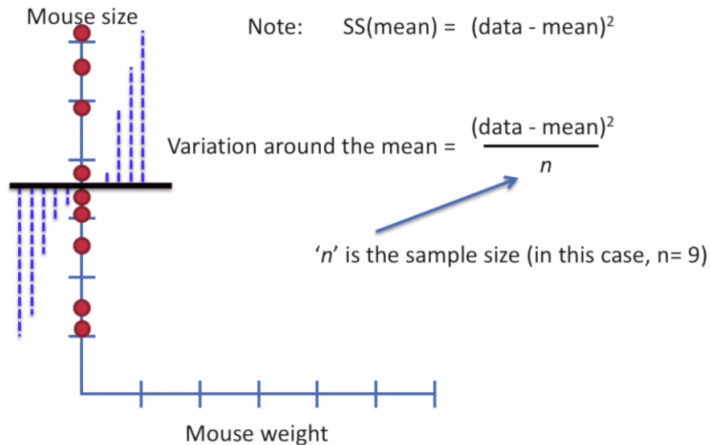


Lineare Regression - Variation um den Mittelwert anschaulich



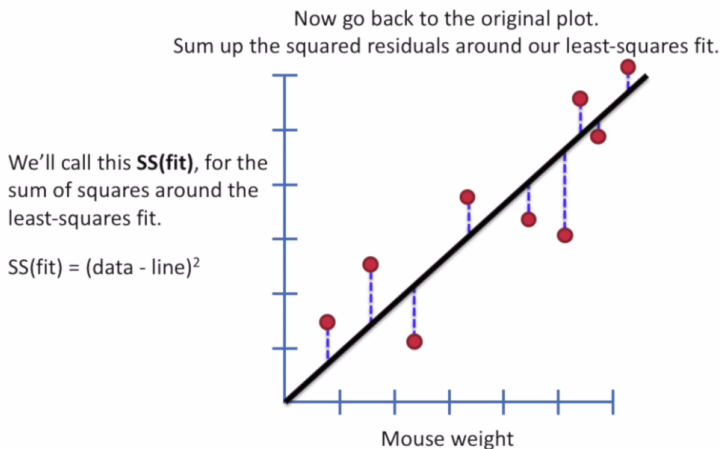
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Residuenminimierung anschaulich



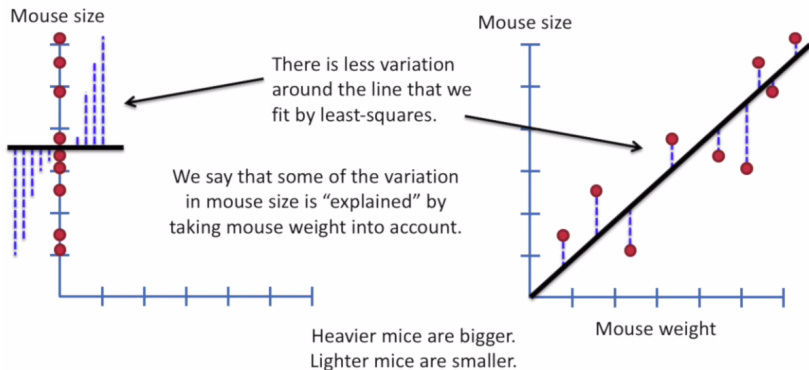
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Mittelwertvarianz versus Residuenminimierung anschaulich



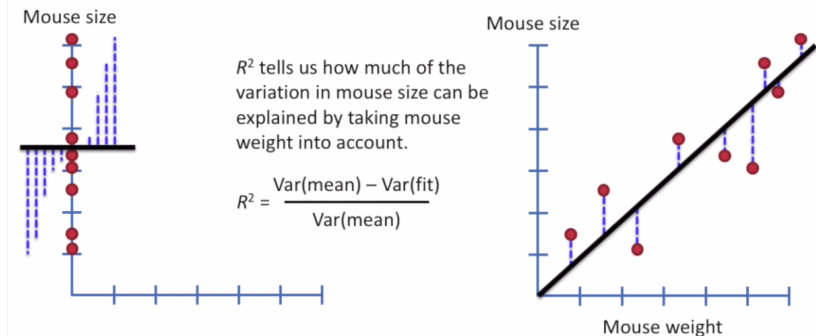
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - R^2 anschaulich



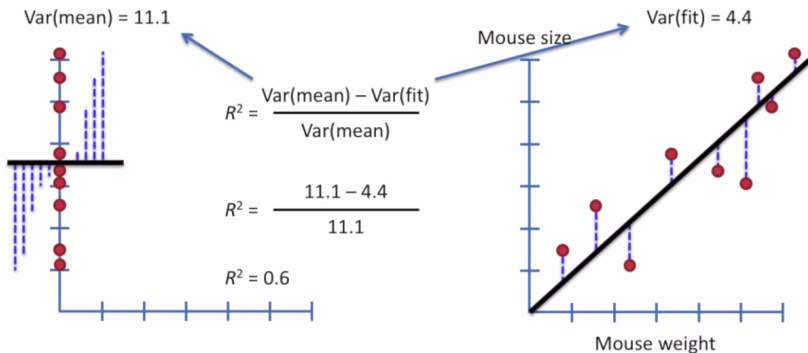
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - R^2 anschaulich



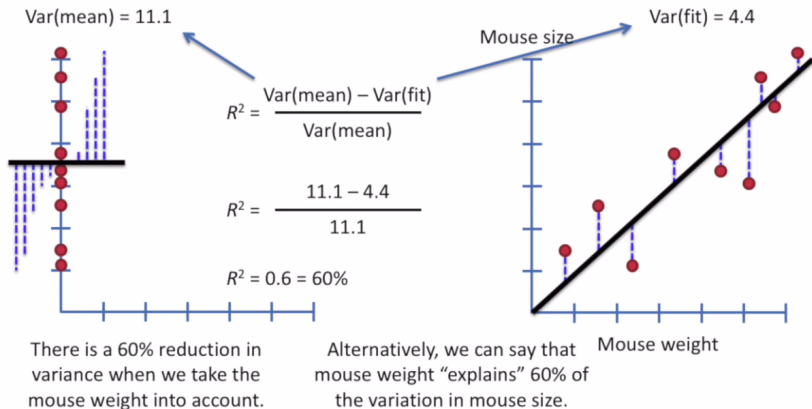
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - R^2 anschaulich



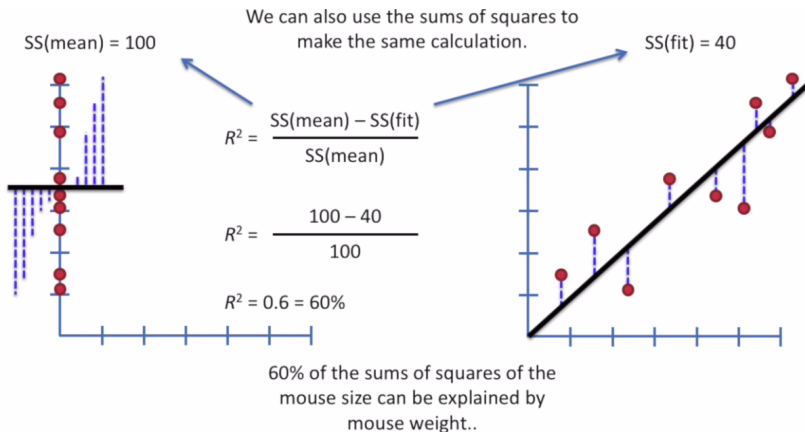
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo, März 2020

Lineare Regression - R^2 anschaulich



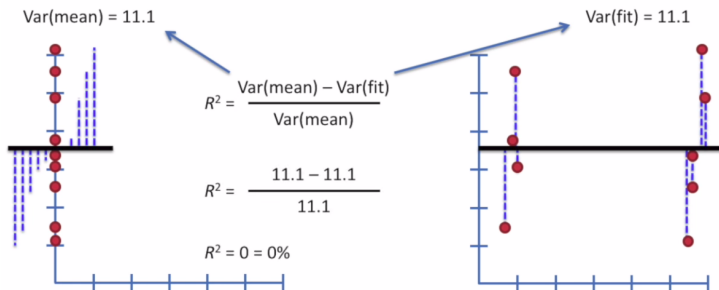
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - R^2 anschaulich



Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - Bestimmtheitsmaß und F-Wert anschaulich



Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - F-Wert anschaulich

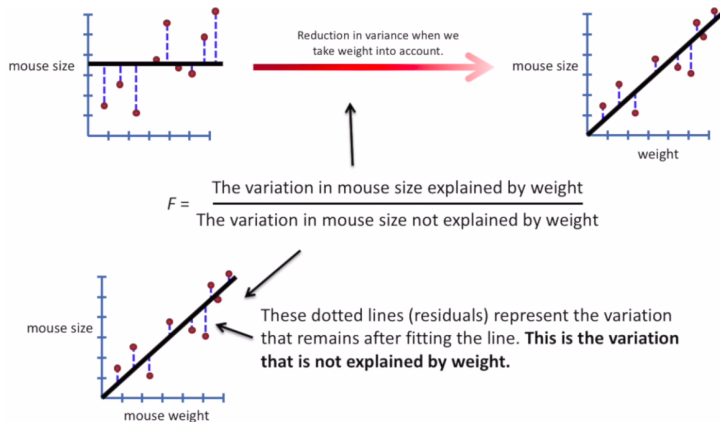
$$R^2 = \frac{\text{The variation in mouse size explained by weight}}{\text{Variation in mouse size without taking weight into account}}$$

$$F = \frac{\text{The variation in mouse size explained by weight}}{\text{The variation in mouse size not explained by weight}}$$

The p -value for R^2 comes from something called " F "

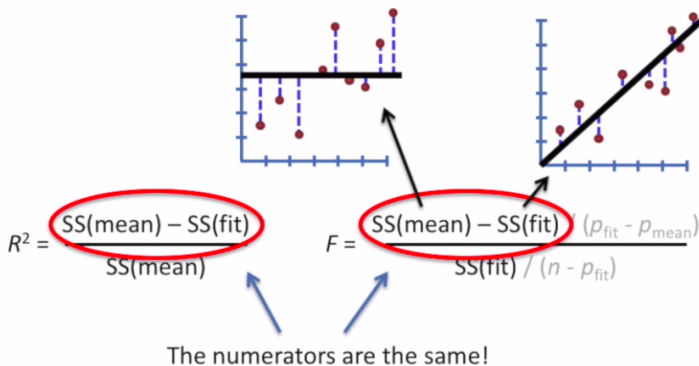
Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - F-Wert anschaulich



Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

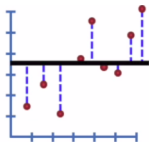
Lineare Regression - F-Wert anschaulich



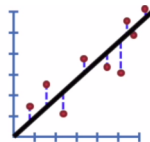
In the our “mouse size and weight example” the numerator is the variation in mouse size explained by weight

Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

Lineare Regression - F-Wert anschaulich



If the "fit" is good, then...



$$F = \frac{\text{The variation explained by the extra parameters in the "fit"}}{\text{The variation not explained by the extra parameters in the "fit"}}$$

→ $\frac{\text{large number}}{\text{small number}}$

$F = \text{really large number}$

How do we turn this number in to a p -value?

Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

sklearn example linear Regression

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

# Load the diabetes dataset
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
x=datasets.load_diabetes()

# Use only one feature
diabetes_X = diabetes_X[:, np.newaxis, 2]

# Split the data into training/testing sets
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]

# Split the targets into training/testing sets
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(diabetes_X_train, diabetes_y_train)

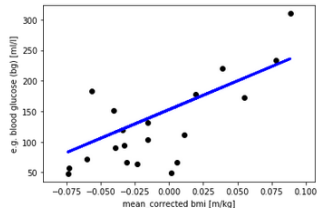
# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)

# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean squared error
print('SME: %.2f'
      % mean_squared_error(diabetes_y_test, diabetes_y_pred))
# The coefficient of determination: 1 is perfect prediction
print('R2-parameter: %.2f'
      % r2_score(diabetes_y_test, diabetes_y_pred))

# Plot outputs
plt.scatter(diabetes_X_test, diabetes_y_test, color='black')
plt.plot(diabetes_X_test, diabetes_y_pred, color='blue', linewidth=3)
plt.ylabel("e.g. blood glucose (bg) [ml/l]")
plt.xlabel("mean_corrected bmi [m/kg]")

plt.show()
```

Coefficients:
[938.23786125]
SME: 2548.07
R2-parameter: 0.47



Definition Multivariate Regression (n-D.)

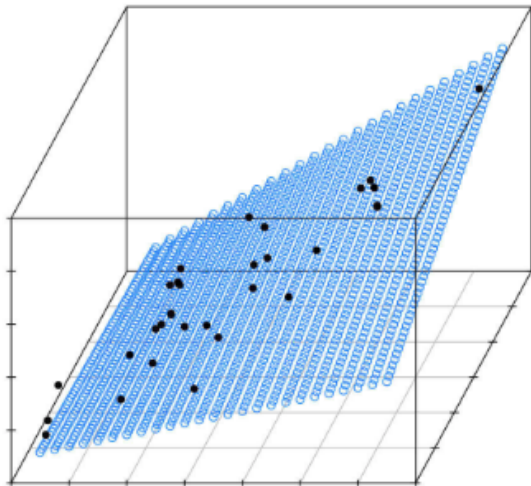
- Bei einer hochdimensionalen Regression wird der Zusammenhang zwischen Zielfunktion und einem Satz von unabhängigen Variablen ($n > 1$) ermittelt.
- Anders als bei der einfachen Regression (1-Dimensional) sind bei der Multiplen Regression keine analytischen Lösungen möglich. Das Minimum der Residuen unter Änderung der Regressionsparameter α_i und β_i wird durch das Gradienten Abstiegsverfahren numerisch bestimmt mit

$$\alpha_{i+1} = \alpha_i + \mu \nabla R_i$$

$$\beta_{i+1} = \beta_i + \mu \nabla R_i$$

und der Learning Rate μ

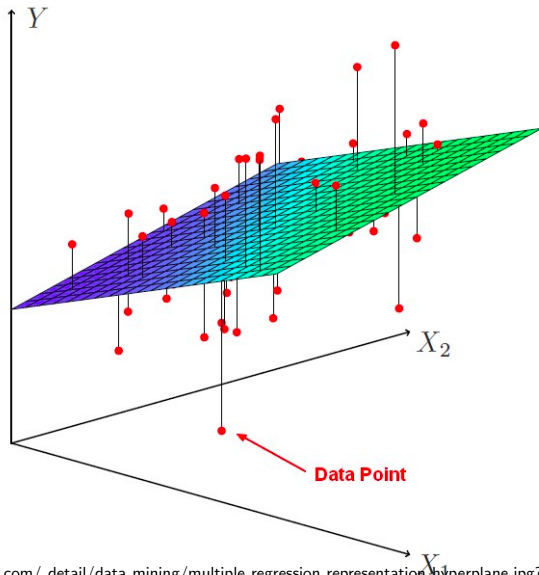
Multiple Regression



Quelle: <https://www.dataquest.io/blog/statistical-learning-for-predictive-modeling-r/>, März 2020

$$y = \beta_1 x_1 + \beta_2 x_2 + \alpha_1 + \alpha_2$$

Multiple Regression

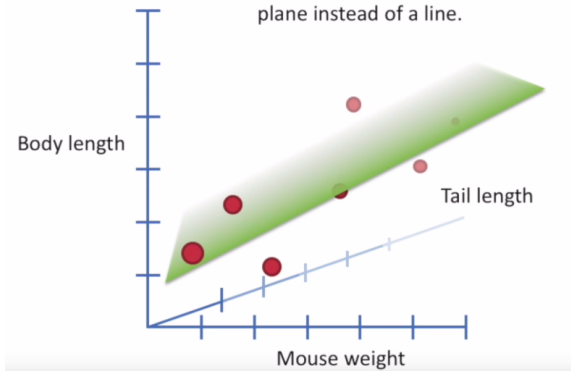


Quelle:

https://gerardnico.com/_detail/data_mining/multiple_regression_representation_hyperplane.jpg?id=data_mining%3Amultiple_re

Lineare Regression - anschaulich

Now we do a least-squares fit. Since we have the extra term in the equation, we fit a plane instead of a line.



Quelle: https://www.youtube.com/watch?v=nk2CQITm_eo , März 2020

- Die Qualitätskennzahlen R^2 und p -Wert werden auf n -Dimensionen erweitert.

https://inria.github.io/scikit-learn-mooc/python_scripts/datasets_california_housing.html

However, we don't know the optimal dosage to give to patients.



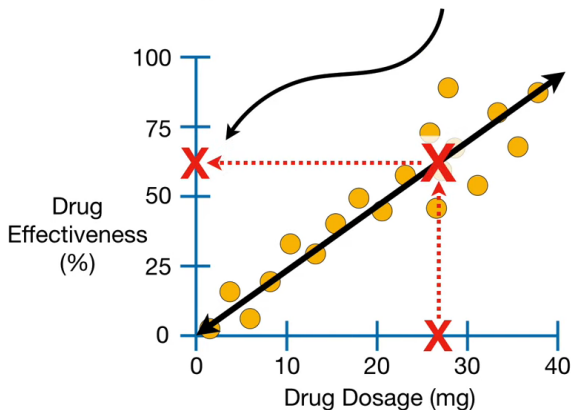
VS.



Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

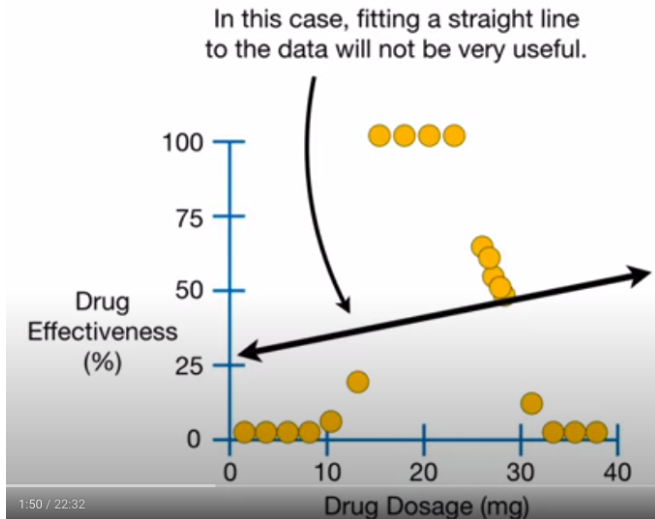
Entscheidungsbaum Regression

...we could use the line to predict that a **27 mg Dose** should be **62% Effective**.



Quelle: <https://www.youtube.com/watch?v=g9c66TUyIZ4>, März 2020

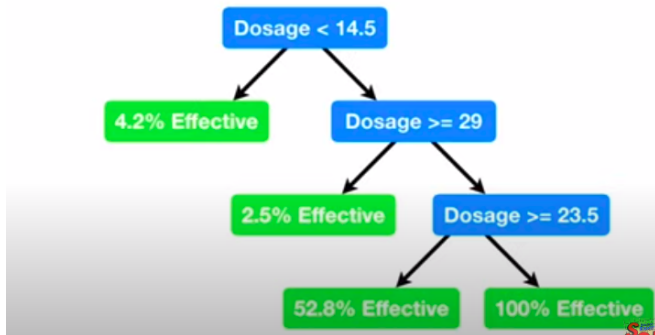
Entscheidungsbaum Regression



Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

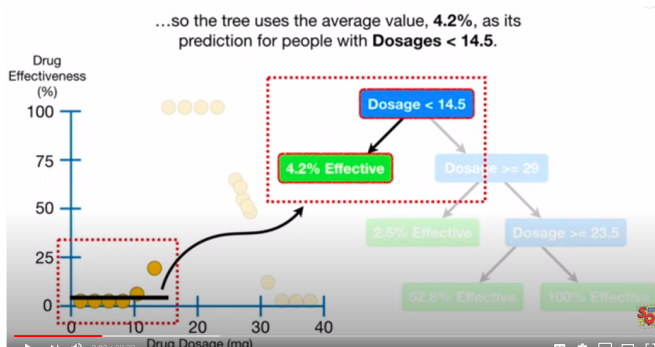
Entscheidungsbaum Regression

Regression Trees are a type of **Decision Tree**.



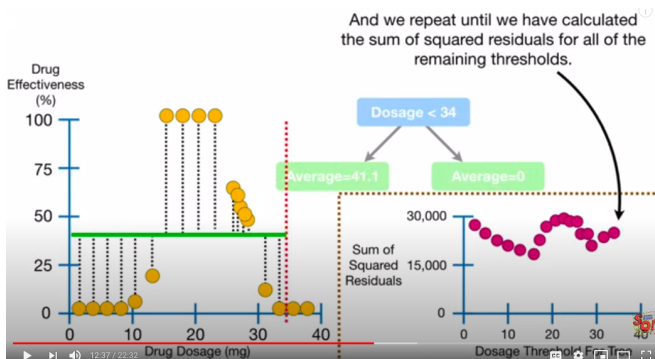
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



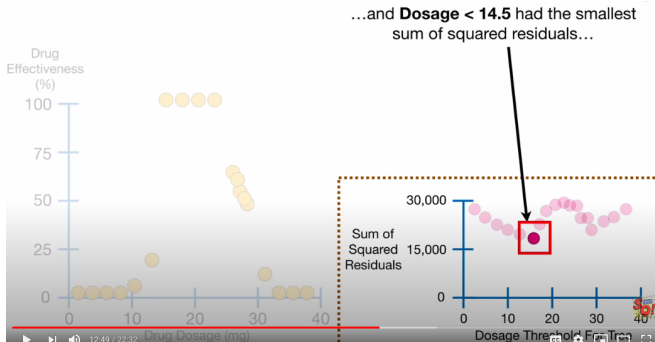
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



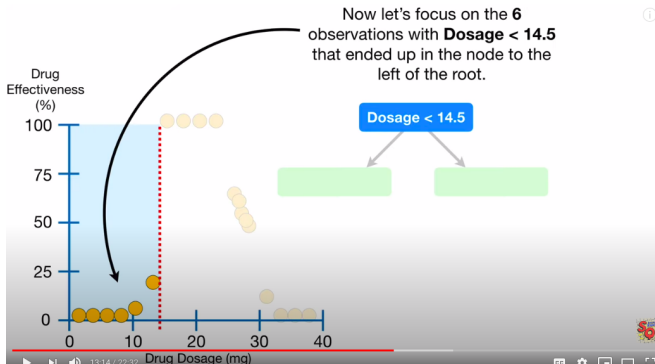
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



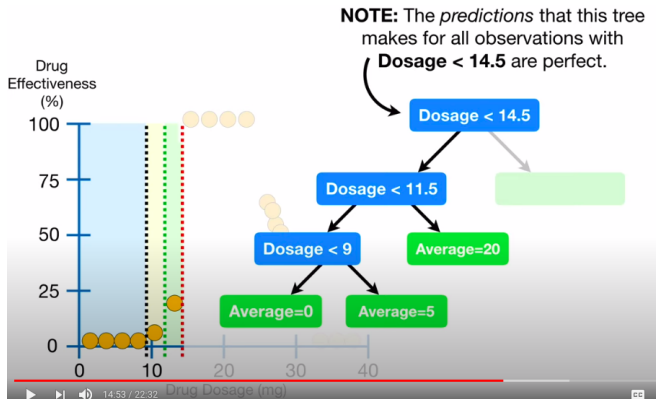
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

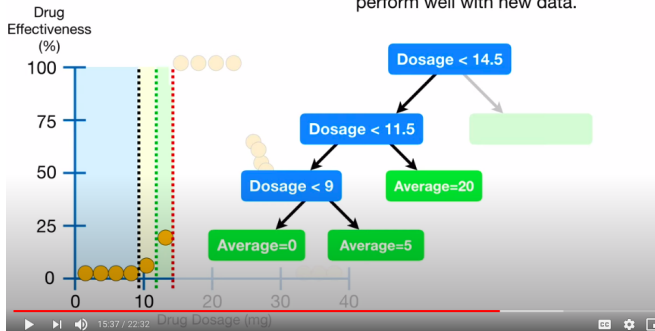
Entscheidungsbaum Regression



Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

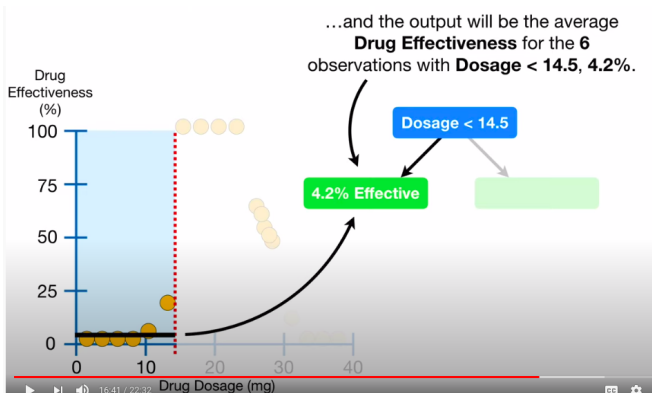
Entscheidungsbaum Regression

When a model fits the training data perfectly, it probably means it is overfit and will not perform well with new data.



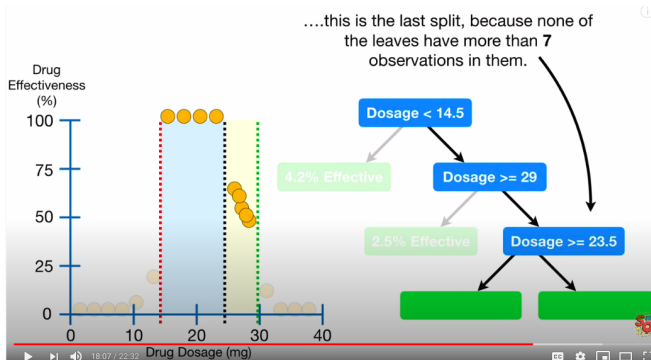
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



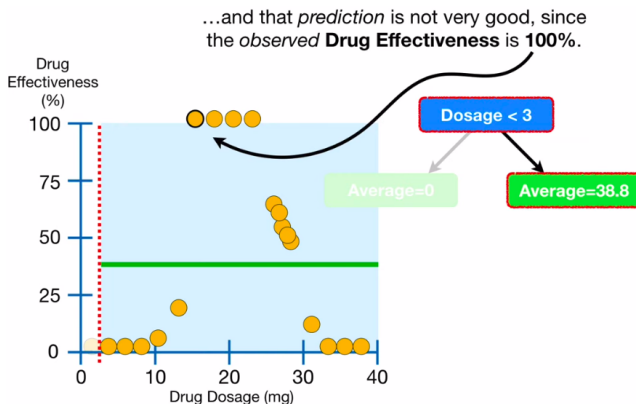
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



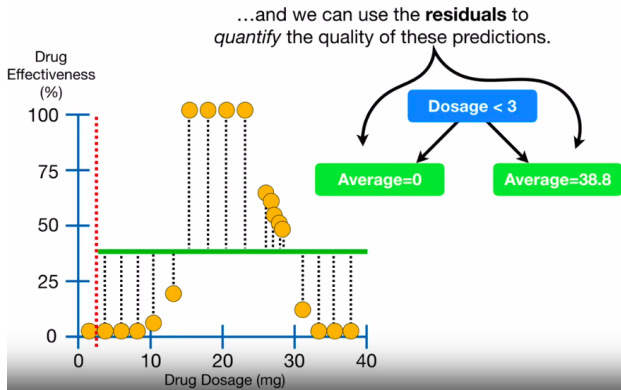
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



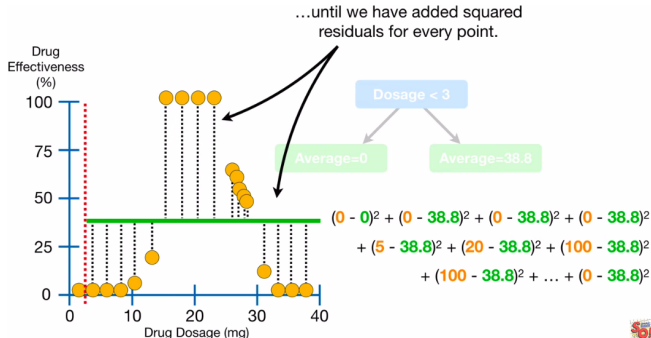
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



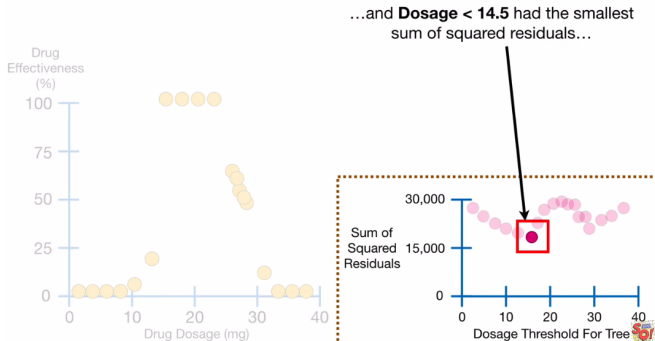
Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

Entscheidungsbaum Regression



Quelle: <https://www.youtube.com/watch?v=g9c66TUylZ4>, März 2020

sklearn Entscheidungsbaum Regression

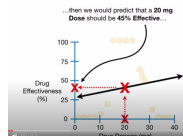
```
# Import the necessary modules and libraries
import numpy as np
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt

# Create a random dataset
rng = np.random.RandomState(1)
X = np.sort(5 * rng.rand(80, 1), axis=0)
y = np.sin(X).ravel()
y[::5] += 3 * (0.5 - rng.rand(16))

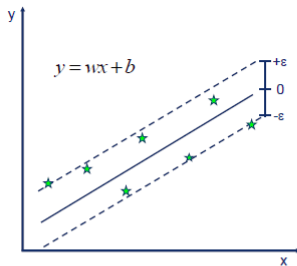
# Fit regression model
regr_1 = DecisionTreeRegressor(max_depth=2)
regr_2 = DecisionTreeRegressor(max_depth=5)
regr_1.fit(X, y)
regr_2.fit(X, y)

# Predict
X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]
y_1 = regr_1.predict(X_test)
y_2 = regr_2.predict(X_test)

# Plot the results
plt.figure()
plt.scatter(X, y, s=20, edgecolor="black",
            c="darkorange", label="data")
plt.plot(X_test, y_1, color="cornflowerblue",
         label="max_depth=2", linewidth=2)
plt.plot(X_test, y_2, color="yellowgreen", label="max_depth=5", linewidth=2)
plt.xlabel("data")
plt.ylabel("target")
plt.title("Decision Tree Regression")
plt.legend()
plt.show()
```



Support Vector Regression



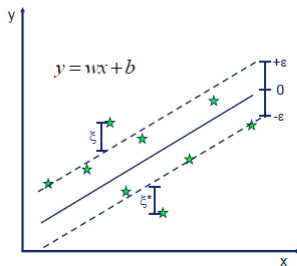
- Solution:

$$\min \frac{1}{2} \|w\|^2$$

- Constraints:

$$y_i - wx_i - b \leq \epsilon$$

$$wx_i + b - y_i \leq \epsilon$$



- Minimize:

$$\frac{1}{2} \|w\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*)$$

- Constraints:

$$y_i - wx_i - b \leq \epsilon + \xi_i$$

$$wx_i + b - y_i \leq \epsilon + \xi_i^*$$

$$\xi_i, \xi_i^* \geq 0$$

Support Vector Regression

```
print(__doc__)

import numpy as np
from sklearn.svm import SVR
import matplotlib.pyplot as plt

# =====
# Generate sample data
X = np.sort(5 * np.random.rand(40, 1), axis=0)
y = np.sin(X).ravel()

# =====
# Add noise to targets
y[:,5] += 3 * (0.5 - np.random.rand(8))

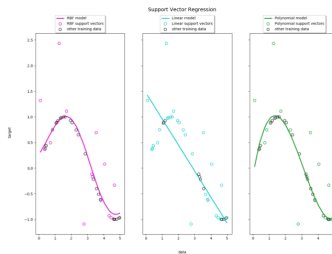
# =====
# Fit regression model
svr_rbf = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=.1)
svr_lin = SVR(kernel='linear', C=100, gamma='auto')
svr_poly = SVR(kernel='poly', C=100, gamma='auto', degree=3, epsilon=.1,
               coef0=1)

# =====
# Look at the results
lw = 2

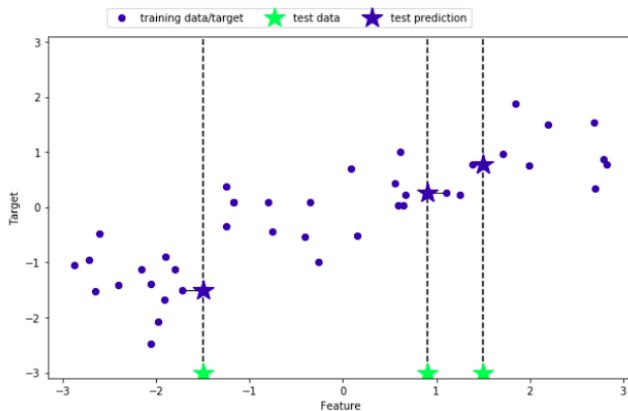
svrs = [svr_rbf, svr_lin, svr_poly]
kernel_label = ['RBF', 'linear', 'Polynomial']
model_color = ['m', 'c', 'g']

fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 10), sharey=True)
for ix, svr in enumerate(svrs):
    axes[ix].plot(X, svr.fit(X, y).predict(X), color=model_color[ix], lw=lw,
                  label='{} model'.format(kernel_label[ix]))
    axes[ix].scatter(X[svr.support_], y[svr.support_], facecolor='none',
                    edgecolor=model_color[ix], s=50,
                    label='{} support vectors'.format(kernel_label[ix]))
    axes[ix].scatter(X[np.setdiff1d(np.arange(len(X)), svr.support_)],
                    y[np.setdiff1d(np.arange(len(X)), svr.support_)],
                    facecolor='none', edgecolor='k', s=50,
                    label='other training data')
    axes[ix].legend(loc='upper center', bbox_to_anchor=(0.5, 1.1),
                    ncol=3, fancybox=True, shadow=True)

fig.text(0.5, 0.04, 'data', ha='center', va='center')
fig.text(0.08, 0.5, 'target', ha='center', va='center', rotation='vertical')
fig.suptitle("Support Vector Regression", fontsize=14)
plt.show()
```

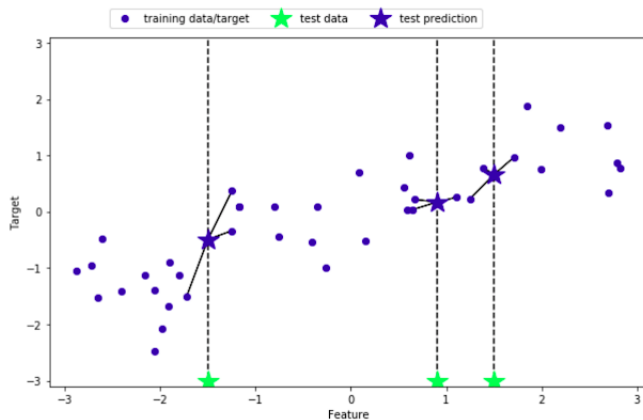


kNN-Regression

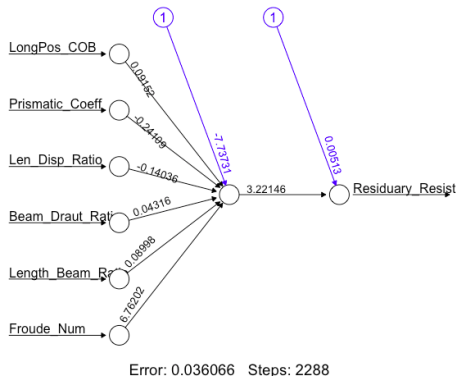


Quelle: <http://the-ml-blogs.blogspot.com/2018/12/5-knn-regression.html>, März 2020

kNN-Regression



Quelle: <http://the-ml-blogs.blogspot.com/2018/12/5-knn-regression.html>, März 2020



Quelle: http://uc-r.github.io/ann_regression, März 2020

- Die Gewichte ω_i enthalten die Information des optimalen Modells mit dem bei der Regression die Residuen minimiert werden.

sklearn ANN-Regression

```
>>> from sklearn.neural_network import MLPRegressor
>>> from sklearn.datasets import make_regression
>>> from sklearn.model_selection import train_test_split
>>> X, y = make_regression(n_samples=200, random_state=1)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y,
...                                                    random_state=1)
>>> regr = MLPRegressor(random_state=1, max_iter=500).fit(X_train, y_train)
>>> regr.predict(X_test[:2])
array([-0.9..., -7.1...])
>>> regr.score(X_test, y_test)
0.4...
```

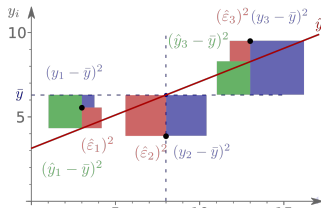
1. Wiederholung des letzten Vorlesungskapitels
2. Zeitreihen (stochastisch/klassisch)
3. Clustering
4. Outlier Detektion
5. Assoziationsanalyse

Regression

- Voraussetzungen:
 - ▶ Homoskedastizität
 - ▶ Normalverteilung der Residuen
- SST (Sum of Squares Total) = SSR (Sum of Squares Regression) + SSE (Sum of Squares Error)

$$\underbrace{\sum_{i=1}^n (y_i - \bar{y})^2}_{SST} = \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{SSR} + \underbrace{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}_{SSE}$$

$\hat{y}$: Regressionspunkte, $\bar{y}$: Mittelwert aller y -Werte, y : Datenpunkte



- nur für den Gebrauch innerhalb der Vorlesung -

- $R^2 = 1 - \frac{SSR}{SST}$
 - ▶ durch unabhängige Variable erklärter Varianzanteil
 - ▶ $R^2 \in [0, 1]$
- $F = \frac{R^2}{1-R^2} \frac{n-k-1}{k}$
 - ▶ Quotient aus erklärtem Varianzanteil R^2 , dividiert durch nicht erklärtem Varianzanteil $1 - R^2$
 - ▶ mit der Stichprobe/Gesamtzahl der Beobachtungen n und der Anzahl an unabhängigen Variablen k
 - ▶ je größer der F-Wert, desto signifikanter ist das Modell

Linear Regression Example

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import numpy as np

# Generate some sample data
X = np.array([[1, 2, 3, 4, 5]]).T
y = np.array([2, 4, 6, 8, 10])

# Create a linear regression model
model = LinearRegression()

# Fit the model to the data
model.fit(X, y)

# Calculate the predicted values
y_pred = model.predict(X)

# Calculate R^2 score
r2 = r2_score(y, y_pred)

# Calculate F-statistic
n = len(y)
k = X.shape[1]
f_statistic = (r2 / k) / ((1 - r2) / (n - k - 1))

print("R^2 score:", r2)
print("F-statistic:", f_statistic)
```

- Decision Trees, SVR, kNN, ANN, ...

Decision Tree Regression Example

```
# Import the necessary modules and libraries
import numpy as np
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt

# Create a random dataset
rng = np.random.RandomState(1)
X = np.sort(5 * rng.rand(80, 1), axis=0)
y = np.sin(X).ravel()
y[::5] += 3 * (0.5 - rng.rand(16))

# Fit regression model
regr_1 = DecisionTreeRegressor(max_depth=2)
regr_2 = DecisionTreeRegressor(max_depth=5)
regr_1.fit(X, y)
regr_2.fit(X, y)

# Predict
X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]
y_1 = regr_1.predict(X_test)
y_2 = regr_2.predict(X_test)

# Plot the results
plt.figure()
plt.scatter(X, y, s=20, edgecolor="black",
            c="darkorange", label="data")
plt.plot(X_test, y_1, color="cornflowerblue",
         label="max_depth=2", linewidth=2)
plt.plot(X_test, y_2, color="yellowgreen", label="max_depth=5", linewidth=2)
plt.xlabel("data")
plt.ylabel("target")
plt.title("Decision Tree Regression")
plt.legend()
plt.show()
```

Support Vector Regression Example

```
from sklearn.svm import SVR
from sklearn.metrics import r2_score
import numpy as np

# Generate some sample data
X = np.array([[1, 2, 3, 4, 5]]).T
y = np.array([2, 4, 6, 8, 10])

# Create an SVR model
model = SVR()

# Fit the model to the data
model.fit(X, y)

# Calculate the predicted values
y_pred = model.predict(X)

# Calculate R^2 score
r2 = r2_score(y, y_pred)

print("R^2 score:", r2)
```

kNN- Regression Example

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score
import numpy as np

# Generate some sample data
X = np.array([[1, 2, 3, 4, 5]]).T
y = np.array([2, 4, 6, 8, 10])

# Create a KNN regression model
model = KNeighborsRegressor(n_neighbors=3)

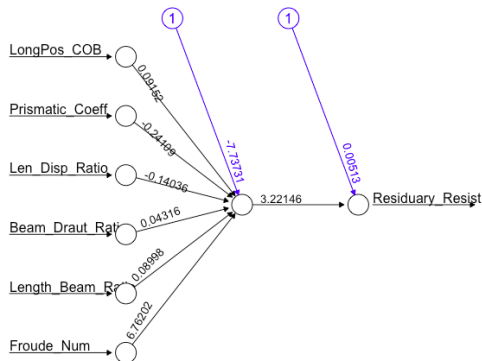
# Fit the model to the data
model.fit(X, y)

# Calculate the predicted values
y_pred = model.predict(X)

# Calculate R^2 score
r2 = r2_score(y, y_pred)

print("R^2 score:", r2)
```

ANN- Regression Example



Error: 0.036066 Steps: 2288

Table of contents

1. Regression
2. Clustering

Definition Clustering

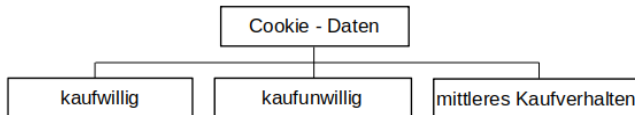
- Die Clusteranalyse/ Ballungsanalyse dient dazu Ähnlichkeitsstrukturen/Cluster/Ballungszentren im Featureraum in Datenbeständen aufzudecken.
- Die Cluster ("natürliche Klassen") sind **vorab unbekannt**. Clusteranalyse gehört zu den "uninformierten Verfahren". Im Gegensatz dazu sind die Klassen bei Klassifikation im Vorfeld festgelegt (Klassifikation zählt zu den "informierten Verfahren").
- Clusteranalyse ist ein wichtiger Vertreter des Unsupervised Learnings. Es gibt auch Clusteralgorithmen des Supervised Learnings.
- Die Cluster können nach ihrer Identifikation z.B. zur automatisierten Klassifizierung, zur Erkennung von Mustern, Bild- oder Marktsegmentierung genutzt werden.

Clusteranalyse dient dazu

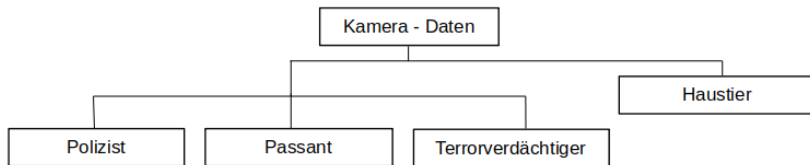
- die Struktur der Daten zu erfassen und aufzudecken (Explorative Analyse).
- Instanzen zu entdecken, die sich ähnlich verhalten und daher ähnlich behandelt werden können (Binning).
- bisher unentdeckte "Klassen" / Datenstrukturen mit ähnlichen Eigenschaften im Datensatz (dargestellt als Punktwolken/Cluster im Feature Raum) aufzudecken.

- E-Commerce: Clustering der verkauften Produkten. Gibt es beispielsweise besonders häufig oder besonders selten gewählte Produkte? (Dies ist sowohl für alle User, Usergruppen als auch einzelne User anwendbar)
- E-Commerce: Einteilung der Käuferschaft in Cluster (Wer ist kaufstark (Alter, Geschlecht, Wohnort, ...)?)
- Produktion: Welche Maschinen einer großen Produktionslinie führen besonders ineffiziente Schritte (langandauernd, schlechte Qualität) und besonders effiziente Schritte aus (schnell, hohe Qualität)?

E-Commerce:

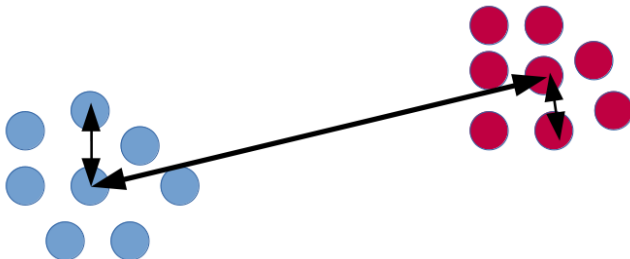


Kamera-Daten:



- Clustering kann auf den folgenden Featureraumeigenschaften erfolgen:
 - ▶ Dichte (z.B. von Datenpunkten ("Datenwolken"))
 - ▶ Abstand (z.B. zwischen Datenpunkten oder über den Abstand der Mittelwerte iterativ angepasster Cluster)
- Manche Clusteralgorithmen benötigen Parameter, die im Vorfeld festgelegt werden müssen. Es gibt z.B. Algorithmen bei denen nicht die Cluster selbst, aber die Anzahl an Clustern im Vorfeld festgelegt werden muss (z.B. k-means).

Distanzbasierte Verfahren (Beispiel zentrumsbasierte Verfahren):



Auswahl von Distanzbegriffen bei unterschiedlichen Skalen

- Binäre Skalen (z.B: männlich, weiblich):

- ▶ Jaccard-Maß (Ähnlichkeitsmaß)

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- Nominale Skalen (z.B.: "Pop", "Rock", "Jazz")

- ▶ Hemming-Distanz (Distanzmaß)

$$\Delta(x, y) = |\{j \in \{1, \dots, n\} | x_j \neq y_j\}|$$

Beispiel: $\Delta(\text{"Hallo"}, \text{"Haus"}) = 3$

- Metrische Skalen (z.B.: 1.234324)

- ▶ Euklidische Metrik (Distanzmaß (z.B. k-means))

$$|\vec{x} - \vec{y}| = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

, Beispiel: $\left| \begin{pmatrix} 1 \\ 2 \end{pmatrix} - \begin{pmatrix} 2 \\ 7 \end{pmatrix} \right| = \sqrt{(1 - 2)^2 + (2 - 7)^2}$

Beispiel Jaccard Maß

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import pairwise_distances
from scipy.spatial.distance import squareform
from itertools import combinations

# Beispiel-Daten
data = [
    {1, 2, 3, 4, 5},
    {4, 5, 6, 7, 8},
    {1, 2, 3},
    {5, 6, 7},
    {8, 9, 10}
]

# Berechnung der Jaccard-Distanzmatrix
dist_matrix = pairwise_distances(data, metric="jaccard")
jaccard_distances = squareform(dist_matrix)

# Hierarchische Clusteranalyse
clustering = AgglomerativeClustering(n_clusters=2, affinity="precomputed", linkage="average")
clustering.fit(jaccard_distances)

# Zuordnung der Datenpunkte zu den Clustern
clusters = clustering.labels_
print("Cluster-Zuordnung:", clusters)
```

Beispiel Hamming Distanz

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import pairwise_distances
from scipy.spatial.distance import squareform

# Beispiel-Daten
data = [
    "01101",
    "10100",
    "01100",
    "00110",
    "11011"
]

# Berechnung der Hamming-Distanzmatrix
dist_matrix = pairwise_distances(data, metric="hamming")
hamming_distances = squareform(dist_matrix)

# Hierarchische Clusteranalyse
clustering = AgglomerativeClustering(n_clusters=2, affinity="precomputed", linkage="average")
clustering.fit(hamming_distances)

# Zuordnung der Datenpunkte zu den Clustern
clusters = clustering.labels_
print("Cluster-Zuordnung:", clusters)
```

Beispiel Euklidische Distanz

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import pairwise_distances
from scipy.spatial.distance import squareform

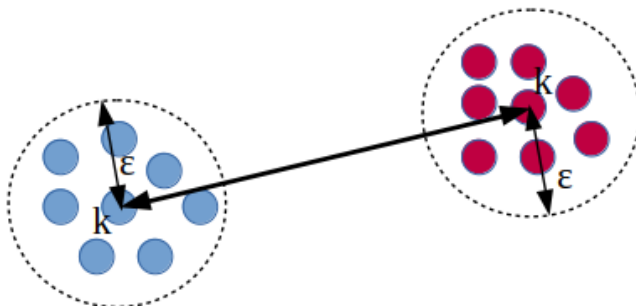
# Beispiel-Daten
data = [
    [2, 3],
    [4, 2],
    [1, 4],
    [6, 4],
    [8, 6]
]

# Berechnung der euklidischen Distanzmatrix
dist_matrix = pairwise_distances(data, metric="euclidean")
euc_distances = squareform(dist_matrix)

# Hierarchische Clusteranalyse
clustering = AgglomerativeClustering(n_clusters=2, affinity="precomputed", linkage="average")
clustering.fit(euc_distances)

# Zuordnung der Datenpunkte zu den Clustern
clusters = clustering.labels_
print("Cluster-Zuordnung:", clusters)
```

Dichtebasierte Verfahren (Beispiel zentrumsbasierte Verfahren):



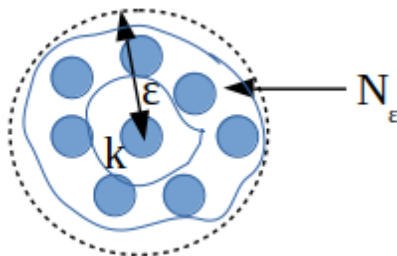
Dichtedefinition Kernobjekt auf Basis des Abstandsbegriffs

- Kernobjekt: ein Kernobjekt $k \in D$ besitzt mindestens n nächste Nachbarn $N_\epsilon(k)$ im Umfeld mit Radius ϵ

$$k \in D : |N_\epsilon(k)| \geq n,$$

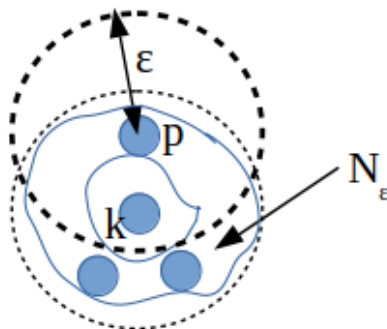
wobei

$$N_\epsilon(k) = \{d_{nn} \in D \mid \text{dist}(k, d_{nn}) \leq \epsilon\}$$



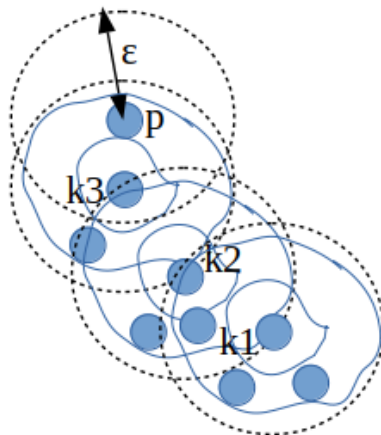
Dichtedefinition Dichte-erreichbarkeit auf Basis des Abstandsbegriffs

- direkt dichte-erreichbar: ein Kernobjekt (zentrales Element) $p \in D$ ist direkt dichte-erreichbar von $k \in D$ bzgl. ϵ , wenn gilt: $p \in N_{\epsilon}(k)$ und k ist ein Kernobjekt (zentrales Element) in D .



Dichtedefinition Dichte-erreichbarkeit auf Basis des Abstandsbegriffs

- dichte-erreichbar: Ein Objekt p ist dichte-erreichbar von k , wenn es eine Kette von direkt dichte erreichbaren Objekten zwischen k und p gibt.



```
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

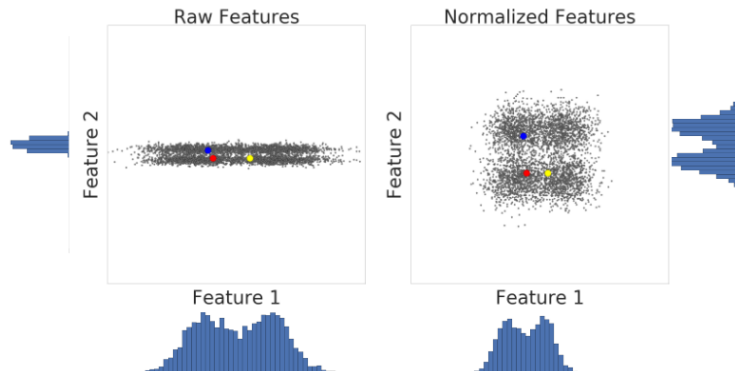
# Generierung der Beispiel-Daten
X, y = make_blobs(n_samples=200, centers=3, random_state=42)

# Anwendung des DBSCAN-Algorithmus
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)

# Visualisierung der Ergebnisse
core_samples_mask = np.zeros_like(labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True

# Plot der Datenpunkte
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(X[core_samples_mask, 0], X[core_samples_mask, 1], c='red', s=50, edgecolor='k')
plt.title('Dichte-Erreichbarkeit mit DBSCAN')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

Clustering Preprocessing



Quelle: <https://developers.google.com/machine-learning/clustering/prepare-data>, Mai 2022

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

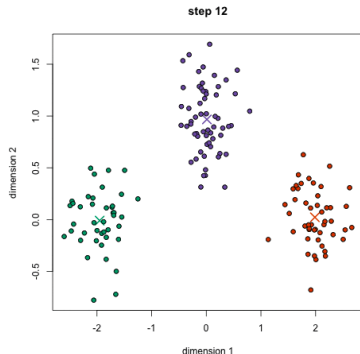
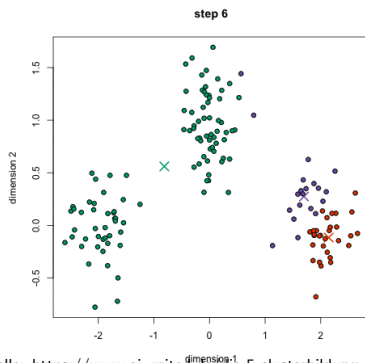
# Generierung der Beispiel-Daten
X, y = make_blobs(n_samples=200, centers=3, random_state=42)

# Daten normieren
scaler = StandardScaler()
X_normalized = scaler.fit_transform(X)

# Anwendung des K-Means-Algorithmus auf die normierten Daten
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_normalized)

# Visualisierung der Ergebnisse
plt.scatter(X_normalized[:, 0], X_normalized[:, 1], c=kmeans.labels_, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='x')
plt.title('Clustering mit normalisierten Daten')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

zentrumsbasiertes, iteratives Verfahren k-means



Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,
März 2020

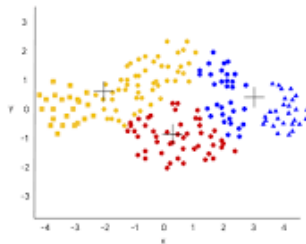
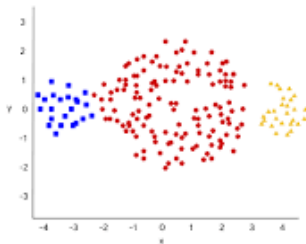
- Ziel von k-Means ist es, den Datensatz so in k (initial bestimmte Anzahl von Clustern) Partitionen zu teilen, dass die Summe der quadrierten Abweichungen von den Cluster-Schwerpunkten μ_i minimal ist. Mathematisch entspricht dies der Optimierung der Funktion:

$$J = \sum_{i=1}^k \sum_{x_j \in S_i} ||x_j - \mu_i||^2$$

Partitionierendes Verfahren: k-Means Algorithmus

- Ziel von k-Means ist es, den Datensatz so in k Partitionen zu teilen, dass die Summe der quadrierten Abweichungen von den Cluster-Schwerpunkten μ_i minimal ist. Mathematisch entspricht dies der Optimierung der Funktion

$$J = \sum_{i=1}^k \sum_{x_j \in S_i} ||x_j - \mu_i||^2$$



Quelle: <https://developers.google.com/machine-learning/clustering/algorithm/advantages-disadvantages>, Februar

Grobschritte:

- Bestimme k , die Anzahl der gewünschten Cluster
- Wähle k beliebige Punkte als Cluster-Zentren aus
- Weise jede Instanz dem nächsten Cluster-Zentrum zu
- Berechne den Mittelpunkt für jeden Cluster und verwende ihn als neues Zentrum
- Weise alle Instanzen wieder dem nächsten Cluster-Zentrum zu
Iteriere, bis alle Cluster stabil sind

Iteratives, Flaches Clustering: Grundlagen k-means (distanz-basiert)

- Jede Instanz $\vec{x}$ im Datenset wird als Vektor mit einem Wert pro Attribut repräsentiert

$$\vec{x} = (x_1, x_2, \dots, x_n)$$

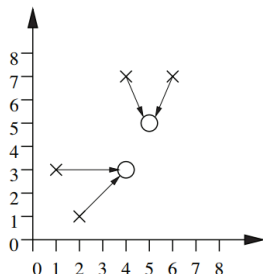
- Die Distanz zwischen zwei Vektoren $\vec{x}$ und $\vec{y}$ ist definiert als (euklidische Distanz):

$$|\vec{x} - \vec{y}| = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

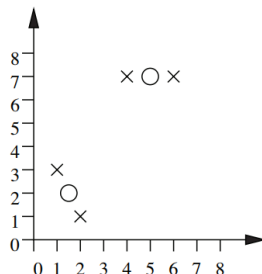
- Der Mittelpunkt $\vec{\mu}$ einer Menge Vektoren c_j ist definiert als:

$$\vec{\mu} = \frac{1}{|c_j|} \sum_{\vec{x} \in c_j} x_j$$

Iteratives, Flaches Clustering: kmeans Beispiel



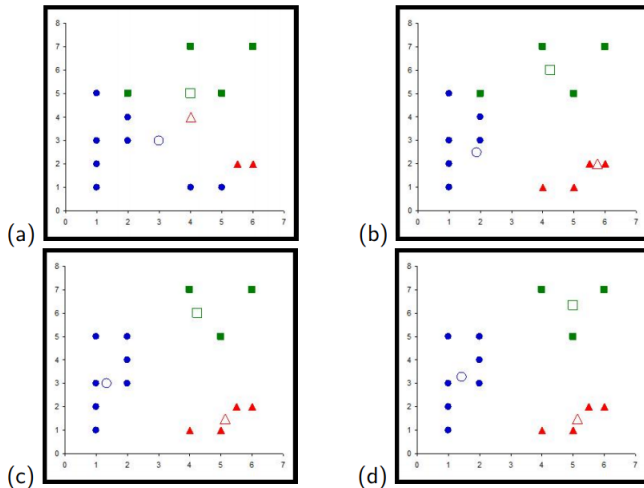
Die Instanzen (Kreuzchen) werden anfangs zum nächsten Cluster-Zentrum (Kreise) zugewiesen



Der Mittelpunkt jedes Cluster wird dann berechnet und als neuen Zentrum verwendet

Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Iteratives, Flaches Clustering: kmeans Beispiel

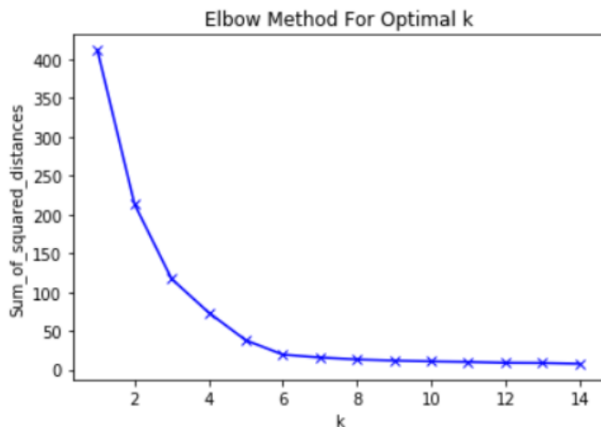


Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Eigenschaften von k-means

- Flaches Clustering-Verfahren
- Effizient bei großen Datenmengen
- Nicht geeignet für Nominaldaten
- Findet nur ein lokales Maximum, keine globales
- Die Cluster hängen stark ab von der initialen Wahl der Clusterzentren ab
- Weiche Version von k-means: EM-Verfahren

Iteratives, Flaches Clustering: Bestimmung der Clusteranzahl k über die elbow Methode



In the plot above the elbow is at $k=5$ indicating the optimal k for this dataset is 5

Quelle: <https://blog.cambridgespark.com/how-to-determine-the-optimal-number-of-clusters-for-k-means-clustering-14f27070048f>, März 2020

Iteratives, Flaches Clustering: Bestimmung der Clusteranzahl k über die elbow Methode

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

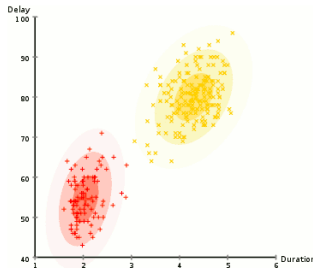
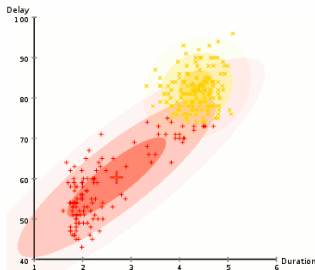
# Generierung der Beispiel-Daten
X, y = make_blobs(n_samples=200, centers=10, random_state=42)

# Durchführung der Elbow-Methode
inertia = []
k_values = range(1, 11)

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

# Visualisierung der Ergebnisse
plt.plot(k_values, inertia, 'bx-')
plt.xlabel('Anzahl der Cluster')
plt.ylabel('Inertia')
plt.title('Elbow-Methode')
plt.show()
```

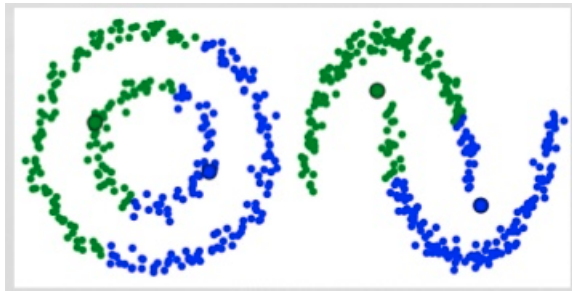
Erwartungs-Maximierungs (EM)-Verfahren: Verallgemeinerung auf Gaußsche Mischungsmodelle (GMM)



Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,
März 2020

- Die naive Mittelwertbildung bei k-means hat Nachteile (ist nur anwendbar auf nahezu isotrope Cluster)
 - Bei dem EM-Verfahren wird eine n-Dimensionale Gaußfunktion der Datenpunkte angenommen mit den Parametern Mittelwert und Standardabweichung. Die zwei Parameter können elliptische Cluster erfassen.
- nur für den Gebrauch innerhalb der Vorlesung -

Grenzen k-Means, Einsatz dichte basiert

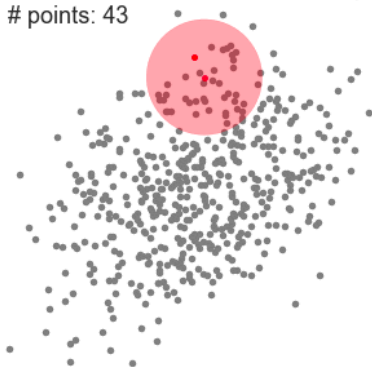


Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,

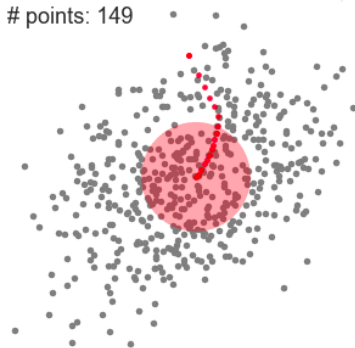
März 2020

zentrumsbasiertes, iteratives Verfahren mean-shift

points: 43



points: 149



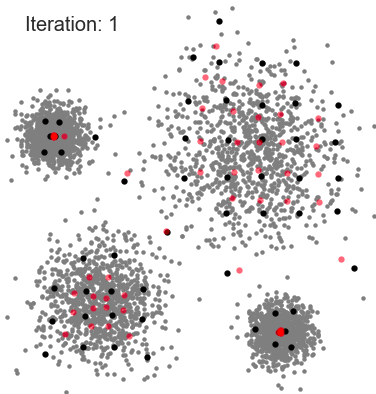
Mean-Shift Clustering for a single sliding window

Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,
März 2020

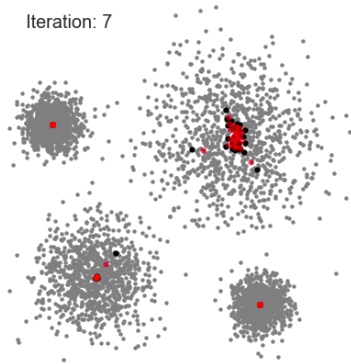
- Das "Sliding-Window" wird iterativ in Richtung des Mittelwertes der Punkte innerhalb des Windows bewegt. Bei heterogen verteilten Punkten wird das Window daher immer weiter Richtung höherer Dichten bewegt.

zentrumsbasiertes, iteratives Verfahren mean-shift

Iteration: 1



Iteration: 7



Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,

März 2020

scikit-learn: mean-shift

```
import numpy as np
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs

# Generate sample data
centers = [[1, 1], [-1, -1], [1, -1]]
X, _ = make_blobs(n_samples=10000, centers=centers, cluster_std=0.6)

# Compute clustering with MeanShift

# The following bandwidth can be automatically detected using
bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=500)

ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(X)
labels = ms.labels_
cluster_centers = ms.cluster_centers_

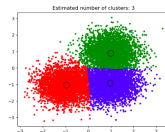
labels_unique = np.unique(labels)
n_clusters_ = len(labels_unique)

print("number of estimated clusters : %d" % n_clusters_)

# Plot result
import matplotlib.pyplot as plt
from itertools import cycle

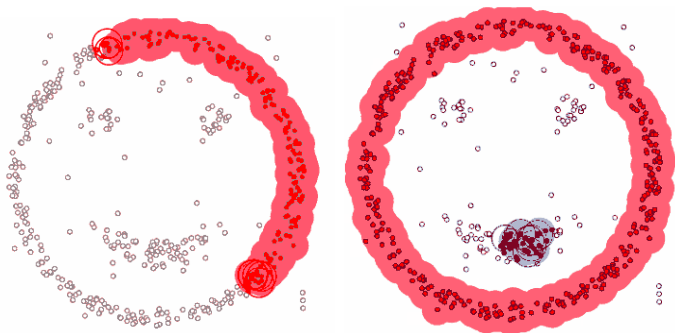
plt.figure(1)
plt.clf()

colors = cycle('bgrcmkybgrcmkybgrcmkybgrcmky')
for k, col in zip(range(n_clusters_), colors):
    my_members = labels == k
    cluster_center = cluster_centers[k]
    plt.plot(X[my_members, 0], X[my_members, 1], col + '.')
    plt.plot(cluster_center[0], cluster_center[1], 'o', markerfacecolor=col,
              markeredgecolor='k', markersize=14)
plt.title('Estimated number of clusters: %d' % n_clusters_)
plt.show()
```



Quelle: www.scikit-learn.org , März 2020

Dichtebasiertes Verfahren DBSCAN



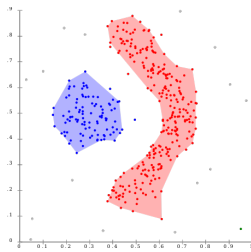
Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,

März 2020

- DBSCAN ist ein auf Dichte basierter Clusterbildung-Algorithmus
- Die Daten werden dichtebasiert abgerastert und als zu einem Cluster zugehörig betrachtet solange Nachbarpunkte noch vorhanden sind.

Dichtebasierte Verfahren - Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

- Objekte, die in einem vorgegebenen Abstand ϵ mindestens k weitere Objekte haben, sind Kernobjekte.
- Zwei Kernobjekte, deren Distanz kleiner als ϵ ist, gehören dabei zum gleichen Cluster.
- Nicht-Kern-Objekte, die nahe einem Cluster liegen, werden diesem als Randobjekte hinzugefügt.
- Objekte, die weder Kernobjekte noch Randobjekte sind, sind Rauschobjekte

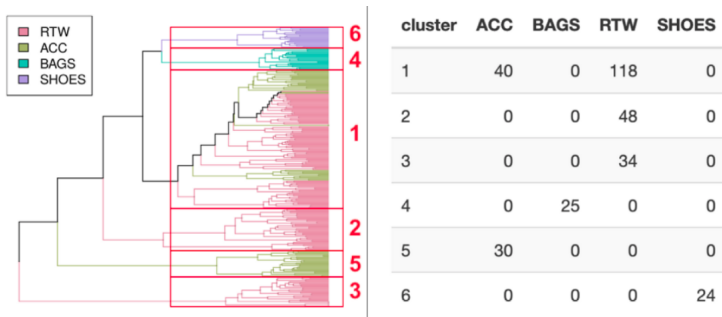


- 1 DBSCAN beginnt mit einem beliebigen Startdatenpunkt, der nicht untersucht wurde. Die Nachbarschaft dieses Punktes wird unter Verwendung einer Entfernung ϵ ; extrahiert (alle Punkte, die innerhalb von ϵ im Abstand liegen, sind Nachbarschaftspunkte).
- 2 Wenn es eine ausreichende Anzahl von Punkten (gemäß minPoints) innerhalb dieser Nachbarschaft gibt, dann beginnt der Clusterbildung-Prozess, und der aktuelle Datenpunkt wird der erste Punkt in dem neuen Cluster. Andernfalls wird der Punkt als Rauschen bezeichnet (dieser rauschende Punkt kann später Teil des Clusters werden). In beiden Fällen wird dieser Punkt als "untersucht" markiert.
- 3 Für diesen ersten Punkt des neuen Clusters werden auch die Punkte in seiner ϵ -Abstandsnachbarschaft Teil des gleichen Clusters. Diese Prozedur, bei der alle Punkte in der ϵ -Nachbarschaft zu demselben Cluster gehören, wird dann für alle neuen Punkte wiederholt, die der Clustergruppe gerade hinzugefügt wurden.

```
>>> from sklearn.cluster import DBSCAN
>>> import numpy as np
>>> X = np.array([[1, 2], [2, 2], [2, 3],
...               [8, 7], [8, 8], [25, 80]])
>>> clustering = DBSCAN(eps=3, min_samples=2).fit(X)
>>> clustering.labels_
array([ 0,  0,  0,  1,  1, -1])
>>> clustering
DBSCAN(eps=3, min_samples=2)
```

Quelle: <https://scikit-learn.org>, Mai 2020

Hierarchisches Verfahren

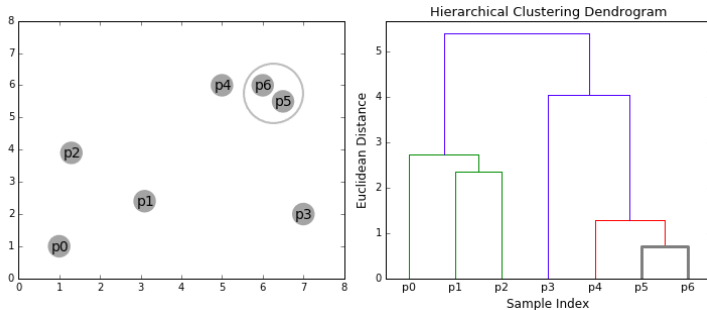


ACC: Accessoires, Bags, Shoes, RTW: ready-to-wear
(Kleidung/keine Maßanfertigung)

Quelle:

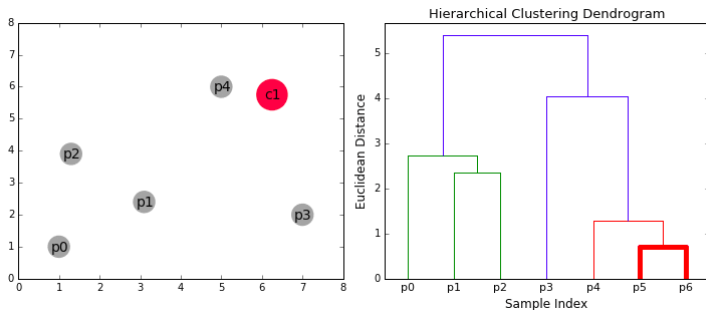
<https://medium.com/ssense-tech/unsupervised-product-clustering-exploring-the-cold-start-problem-8053ef04bac9>,
März 2020

Hierarchisches Verfahren



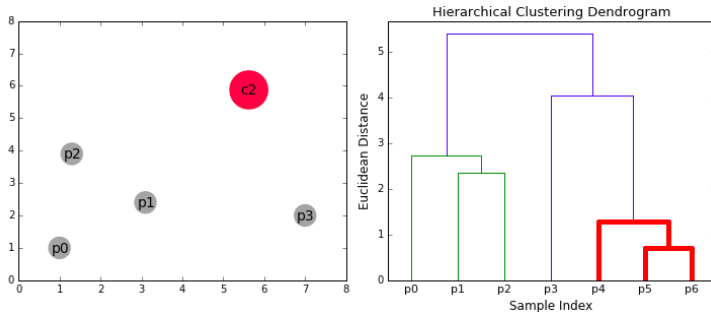
Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>, März 2020

Hierarchisches Verfahren



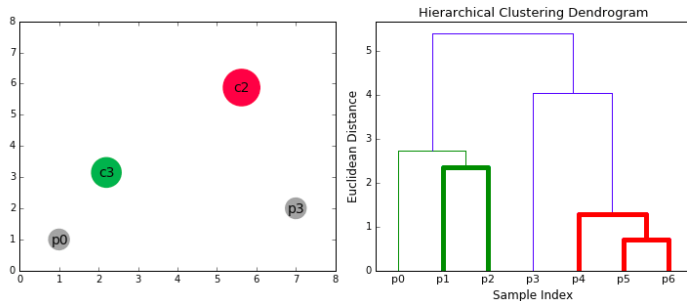
Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,
März 2020

Hierarchisches Verfahren



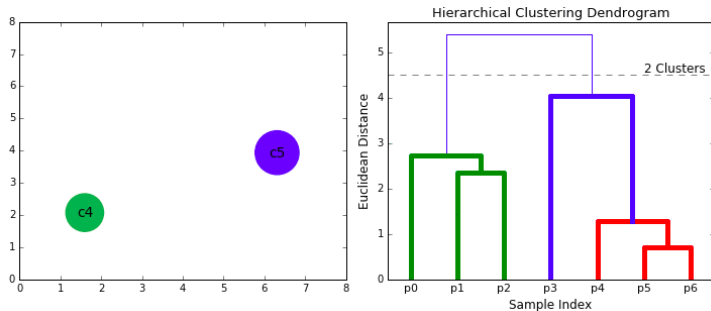
Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>, März 2020

Hierarchisches Verfahren



Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,
März 2020

Hierarchisches Verfahren



Quelle: <https://www.ai-united.de/die-5-clusterbildung-algorithmen-die-jeder-datenwissenschaftler-kennen-muss/>,
März 2020

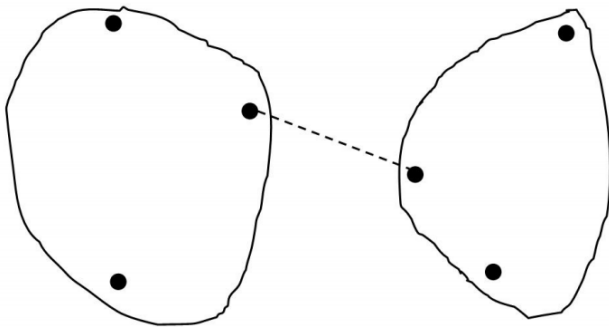
Beim hierarchischen Clustering wird ein Baum generiert, der beschreibt, wie stark sich die verschiedenen Instanzen / Gruppen von Instanzen ähneln.

- Bottom-up agglomeratives Clustering Jede Instanz ist ein Cluster. Gruppiere die zwei ähnlichsten Cluster zu einem neuen Cluster.
- Top-down divisives Clustering fange an mit Cluster, das alle Instanzen enthält, und teile das am wenigsten kohärente Cluster in zwei Cluster auf.

Berechnung der Ähnlichkeit

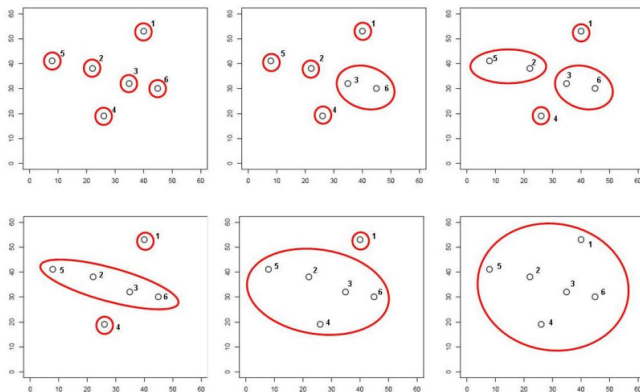
- Single Link: Distanz der ähnlichsten Instanzen zweier Cluster
- Complete Link: Distanz der entferntesten Instanzen zweier Cluster
- Group-Average: durchschnittliche Distanz der Instanzen zweier Cluster

Ähnlichkeitsfunktion: Single Link Clustering



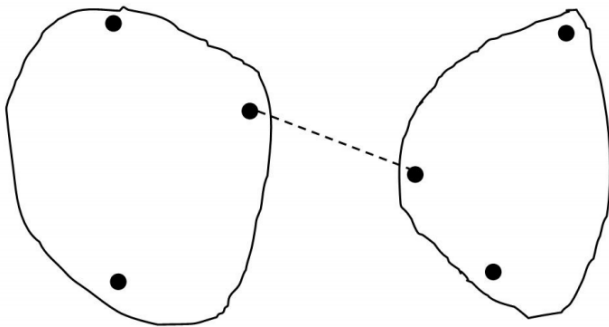
Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Ähnlichkeitsfunktion: Single Link Clustering



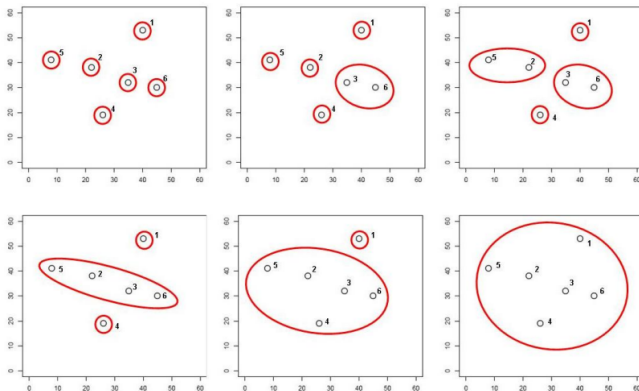
Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Ähnlichkeitsfunktion: Complete Link Clustering



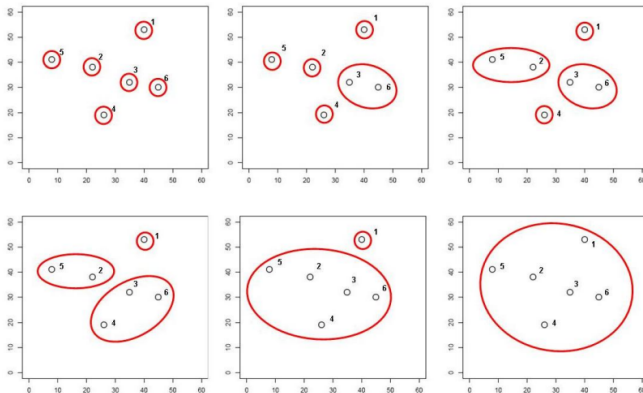
Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Ähnlichkeitsfunktion: Complete Link Clustering



Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Ähnlichkeitsfunktion: Group Link Clustering

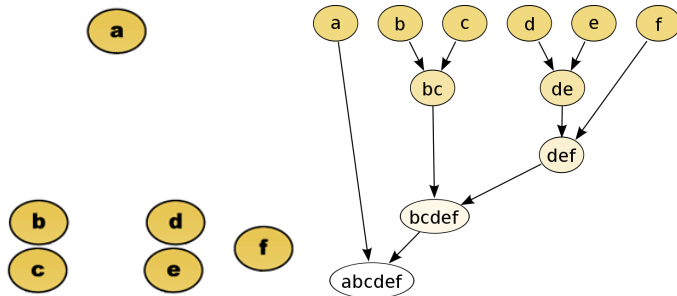


Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Vergleich Ähnlichkeitsfunktionen beim Clustering

- single link führt zu elongierten Clustern
- complete link verhindert elongierte Cluster
- group-average beschreibt "isotropeste" Clustering um einen Mittelpunkt.
- Welche Funktion angebracht ist, hängt von den Daten / der Anwendung ab.

hierarchisches Clustering



Quelle: wikipedia.org, März 2020

Hierarchisches Clustering

- gut für detaillierte Datenanalyse
- mehr Information als flaches Clustering
- bester Algorithmus hängt von der Anwendung ab
- weniger effizient als flaches Clustering

Flaches Clustering

- gut, wenn Effizienz wichtig ist
- k-means Clustering ist ein einfacher Algorithmus dafür sind Resultate oft ausreichend.
- Voraussetzung für k-means clustering: Daten können in Euklidischen Raum dargestellt werden.
- Alternativ: EM Algorithmus

- Als hierarchische Clusteranalyse bezeichnet man eine bestimmte Familie von distanzbasierten Verfahren zur Clusteranalyse.
- Cluster bestehen hierbei aus Objekten, die zueinander eine geringere Distanz aufweisen als zu den Objekten anderer Cluster.
- Dabei wird eine Hierarchie von Clustern aufgebaut: auf der einen Seite ein Cluster, der alle Objekte enthält, und auf der anderen Seite so viele Cluster, wie man Objekte hat, d. h., jedes Cluster enthält genau ein Objekt. Hierarchische Clusteranalyse unterteilt sich in
 - ▶ divisive Clusterverfahren (zunächst werden alle Objekte als zu einem Cluster gehörig angesehen ("Top-Down"))
 - ▶ agglomerative Clusterverfahren (zunächst bildet jedes Objekt einen Cluster ("Bottom-Up"))

Hierarcisches Clustering - divisiv - Divisive Analysis Clustering (DIANA)

1. Starte mit einem Cluster, der alle Beobachtungen enthält.
2. Berechne den Durchmesser aller Cluster. Der Durchmesser ist die maximale Distanz oder Unähnlichkeit aller Objekte innerhalb des Clusters.
3. Der Cluster mit dem größten Durchmesser wird in zwei Cluster geteilt.
4. Dazu wird das Objekt in dem Cluster bestimmt, das die größte durchschnittliche Distanz oder Unähnlichkeit zu allen anderen Objekten hat. Es bildet den Kern der "Splittergruppe".
5. Jedes Objekt, das näher an der Splittergruppe liegt als an den restlichen Objekten, wird nun der Splittergruppe zugeordnet.
6. Die Schritte 2–5 werden solange wiederholt, bis alle Cluster nur noch ein Objekt enthalten.

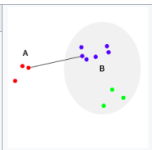
- Single Linkage: Die Cluster, deren nächste Objekte die kleinste Distanz oder Unähnlichkeit haben, werden fusioniert.

Single-Linkage^{[1][2][3][4]}

Minimaler Abstand aller Elementpaare aus den beiden Clustern

$$D_{\text{single-linkage}}(A, B) := \min_{a \in A, b \in B} \{d(a, b)\}.$$

Dieses Verfahren neigt zur Kettenbildung.



Quelle: www.wikipedia.org, Februar 2022

- Ward: Die Cluster, die den kleinsten Zuwachs der totalen Varianz haben, werden fusioniert..

Ward's minimum variance^[8]

Zunahme der Varianz beim Vereinigen von A und B

$$D_{\text{Ward}}(A, B) := \frac{d(\bar{a}, \bar{b})^2}{1/|A| + 1/|B|}$$

wobei $\bar{a}$ das Zentrum des Clusters A sei, $\bar{b}$ das des Clusters B . Dieses Verfahren neigt zur Bildung von gleich großen Clustern.

Quelle: www.wikipedia.org, Februar 2022

```
>>> from sklearn.cluster import AgglomerativeClustering
>>> import numpy as np
>>> X = np.array([[1, 2], [1, 4], [1, 0],
...               [4, 2], [4, 4], [4, 0]])
>>> clustering = AgglomerativeClustering().fit(X)
>>> clustering
AgglomerativeClustering()
>>> clustering.labels_
array([1, 1, 1, 0, 0, 0])
```

linkage : {"ward", "complete", "average", "single"}, default="ward"

Which linkage criterion to use. The linkage criterion determines which distance to use between sets of observation. The algorithm will merge the pairs of cluster that minimize this criterion.

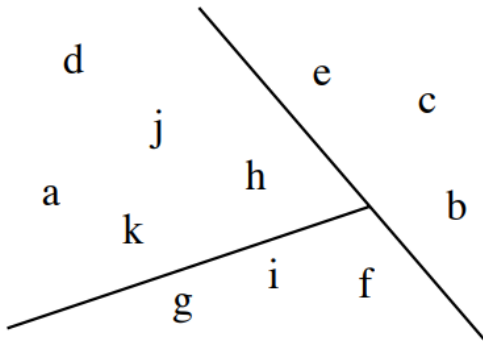
- ward minimizes the variance of the clusters being merged.
- average uses the average of the distances of each observation of the two sets.
- complete or maximum linkage uses the maximum distances between all observations of the two sets.
- single uses the minimum of the distances between all observations of the two sets.

Quelle: <https://scikit-learn.org>, Mai 2020

Je nach Cluster Definition werden Daten unterschiedlich strukturiert (als zu einem Cluster zugehörig oder nicht eingeordnet).

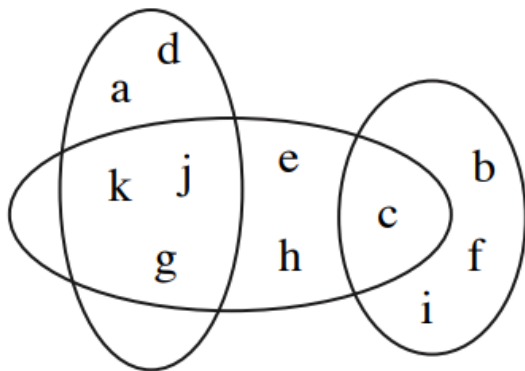
- **iterativ:** Der Algorithmus beginnt mit einer Anfangsmenge von Clustern und verbessert diese immer weiter. Die Anzahl an Clustern bleibt erhalten.
- **hierarchisch:** hierarchische Algorithmen generieren eine Hierarchie von Clustern, sodass es verschiedene Granularitäten gibt.
- **flach:** Bei flachen Algorithmen gibt es nur ein Granularitätslevel und alle Cluster sind gleichwertig.

- **hartes Clustering:** Jedes Objekt gehört ganz oder gar nicht zu einem Cluster
- **weiches Clustering:** jedes Objekt gehört zu einem bestimmten Anteil zu einem Cluster
- **strikt partitionierendes Clustering:** jedes Objekt gehört genau zu einem Cluster
- **strikt partitionierendes Clustering mit Ausreißern:** ein Objekt kann auch zu keinem Cluster gehören.
- **überlappendes/disjunktives Clustering:** ein Objekt kann zu mehreren Clustern gehören.



Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Disjunktes Clustering



Quelle: <http://www.coli.uni-saarland.de/courses/mathe3/SS12/Vorlesungen/k-means.pdf>, März 2020

Instanz	P(cluster 1)	P(cluster 2)	P(cluster 3)
a	0.4	0.1	0.5
b	0.1	0.8	0.1
c	0.3	0.3	0.4
d	0.1	0.1	0.8
e	0.4	0.2	0.4
f	0.1	0.4	0.5
g	0.7	0.2	0.1
h	0.5	0.4	0.1

- Für jeden Cluster wird eine Wahrscheinlichkeit angegeben, dass eine Instanz ihm zugeordnet wird.

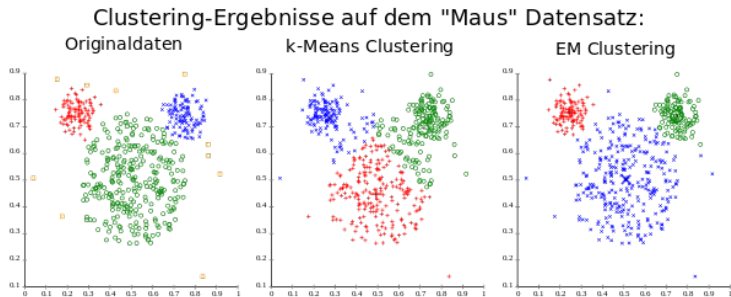
Was in einem Datenset als Cluster definiert wird, muss vor jeder Analyse festgelegt werden.

- k-Means: Cluster wird durch Mittelpunkt beschrieben
- EM-Algorithmus: Cluster wird durch Mittelpunkt und Kovarianzmatrix beschrieben
- DBSCAN: Cluster werde durch ihre Dichte beschrieben

Je nach Cluster Definition werden Daten unterschiedlich strukturiert (als zu einem Cluster zugehörig oder nicht eingeordnet).

- Die Struktur der Daten gibt vor welche Clusterdefinition und welches Clusterverfahren am geeignetsten ist.
- Das Auffinden der optimalen Clusterdefinition/des optimalen Clusterverfahrens stellt eine besondere Herausforderung des Clusterings dar.
- In der Praxis werden häufig mehrere Clusterverfahren/-definitionen auf ein Datensatz angewandt um zu erkennen welches Verfahren am geeignetsten ist.

Standard Mouse-Clusteraset: Vergleich Cluster Definitionen

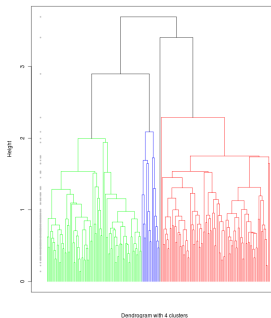
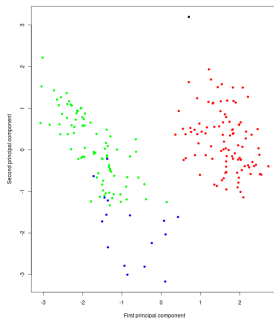


Quelle: wikipedia.org, Februar 2020:

- Das Maus-Datenset wurde von einer Normalverteilung erzeugt.
- Der Cluster-Begriff in k-Means eignet sich nicht gut, um die Cluster zu identifizieren.
- Der EM-Algorithmus, der auf Gaußverteilung beruht, eignet sich hingegen gut.

- (zentrumbasierte) partitonierende Verfahren (häufig iterativ), z.B. k-Means, EM-Verfahren
- dichte-basierte Verfahren
- hierarchische Verfahren
- kombinierte Verfahren

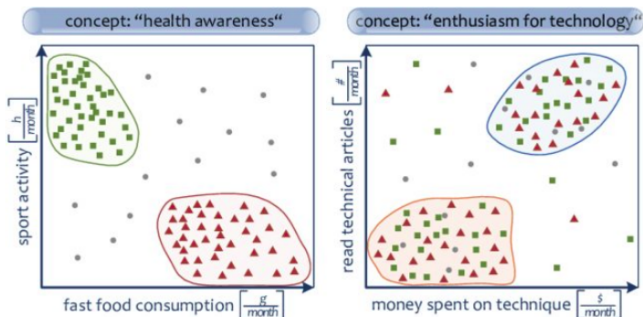
average linkage



- Die zu clusterenden Objekte können auch als Knoten eines Graphs aufgefasst werden und die gewichteten Kanten geben Distanz oder Unähnlichkeit wieder.
- Graphenreduktion: In einem ersten Schritt wird der Graph reduziert. Ziel ist es dabei, alle Kanten mit zu großen Gewichten aus dem Graphen zu entfernen.
- Die Laplace-Matrix, eine spezielle Transformierte der Adjazenzmatrix (Matrix der Ähnlichkeit zwischen allen n Objekten), hat bei k Zusammenhangskomponenten (Clustern) den Eigenwert Null mit der Vielfachheit k .
- Zum Clustern analysiert man die kleinsten Eigenwerte und -vektoren der Laplace-Matrizen

Subspace-Cluster

- Subspace-Cluster zeigen nur bei Projektionen auf bestimmte Feature-Achsen Ähnlichkeiten
- Subspace-Cluster können in Achsenparallelen Clustern und beliebig orientierten Correlation Clustern unterteilt werden

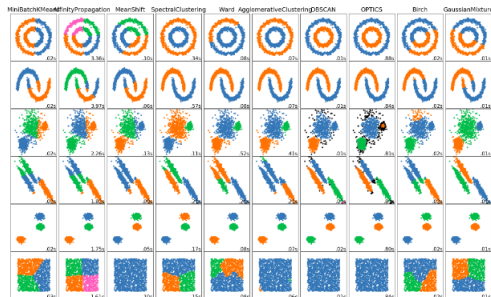


Quelle:

https://www.researchgate.net/publication/224212871_MCExplorer_Interactive_Exploration_of_Multiple_Subspace_Clustering_Sc

März 2020

sklearn - Clustering Übersicht



A comparison of the clustering algorithms in scikit-learn

Method name	Parameters	Scalability	Usecase	Geometry (metric used)
K-Means	number of clusters	Very large n_{samples} , medium n_{clusters} with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters	Distances between points
Affinity propagation	damping, sample preference	Not scalable with n_{samples}	Many clusters, uneven cluster size, non-flat geometry	Graph distance (e.g. nearest-neighbor graph)
Mean-shift	bandwidth	Not scalable with n_{samples}	Many clusters, uneven cluster size, non-flat geometry	Distances between points
Spectral clustering	number of clusters	Medium n_{samples} , small n_{clusters}	Few clusters, even cluster size, non-flat geometry	Graph distance (e.g. nearest-neighbor graph)
Ward hierarchical clustering	number of clusters or distance threshold	Large n_{samples} and n_{clusters}	Many clusters, possibly connectivity constraints	Distances between points
Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large n_{samples} and n_{clusters}	Many clusters, possibly connectivity constraints, non-Euclidean distances	Any pairwise distance
DBSCAN	neighborhood size	Very large n_{samples} , medium n_{clusters}	Non-flat geometry, uneven cluster sizes	Distances between nearest points
OPTICS	minimum cluster membership	Very large n_{samples} , large n_{clusters}	Non-flat geometry, uneven cluster sizes, variable cluster density	Distances between points
Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation	Mahalanobis distances to centers
Birch	branching factor, threshold, optional global clusterer	Large n_{clusters} and n_{samples}	Large dataset, outlier removal, data reduction	Euclidean distance between points